

Edson Junio Rodrigues Lobo

Curso Prático de

MySQL



Abril, 2008

© 2008 by Digerati Books

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. Nenhuma parte deste livro, sem autorização prévia por escrito da editora, poderá ser reproduzida ou transmitida sejam quais forem os meios empregados: eletrônicos, mecânicos, fotográficos, gravação ou quaisquer outros.

Diretor Editorial

Luis Matos

Preparação dos Originais

Jeferson Ferreira

Coordenação Editorial

Renata Miyagusku

Revisão

Guilherme Laurito Summa

Assistência Editorial

Carolina Evangelista

Diagramação

Fabiana Pedrozo e Stephanie Lin

Projeto Gráfico

Daniele Fátima

Capa

Marcos Mazzei

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

L799c Lobo, Edson.

Curso Prático de MySQL / Edson Lobo.
– São Paulo : Digerati Books, 2008.
128 p.

ISBN 978-85-60480-98-2

1. MySQL (Linguagem de programação de computadores). 2. Banco de dados. I. Título.

CDD 005.13

Universo dos Livros Editora Ltda.

Rua Tito, 1.609

CEP 05051-001 • São Paulo/SP

Telefone: (11) 3648-9090 • Fax: (11) 3648-9083

www.universodoslivros.com.br

e-mail: editor@universodoslivros.com.br

Conselho Administrativo: Alessandro Gerardi, Alessio Fon Melozo,
Luis Afonso G. Neira, Luis Matos e William Nakamura.

Sumário

Parte 1

Introdução	5
Capítulo 1 – O que este livro aborda?	6
Capítulo 2 – Bancos de dados	7
Capítulo 3 – Modelagem de dados	9
Capítulo 4 – Criando um MER.....	13
Capítulo 5 – SGBD's (Sistemas de Gerenciamento de Bancos de Dados).....	16

Parte 2

O banco de dados MySQL.....	19
Capítulo 6 – Histórico	20
Capítulo 7 – Licença de uso	22
Capítulo 8 – Instalando e configurando o MySQL	23
Capítulo 9 – Tipos de dados do MySQL.....	35
Capítulo 10 – HeidiSQL: ferramenta gráfica <i>open-source</i> de gerenciamento do MySQL.....	37

Parte 3

MySQL básico	45
Capítulo 11 – Revisando SQL.....	46
Capítulo 12 – Criando e ativando um banco de dados no MySQL	53

Capítulo 13	– Excluindo um banco de dados	55
Capítulo 14	– Criando tabelas	57
Capítulo 15	– Excluindo tabelas	61
Capítulo 16	– Bloqueando e desbloqueando tabelas	63
Capítulo 17	– Modificando uma tabela existente	65
Capítulo 18	– Inserindo novos dados nas tabelas.....	67
Capítulo 19	– Operadores	70
Capítulo 20	– Excluindo tuplas de tabelas	72
Capítulo 21	– Visualizando dados existentes	74
Capítulo 22	– Criando condições para visualização.....	77
Capítulo 23	– Gerando consultas compostas.....	80
Capítulo 24	– Realizando consultas ordenadas.....	85
Capítulo 25	– Utilizando funções predefinidas.....	86
Capítulo 26	– Fazendo consulta em várias tabelas	88

Parte 4

MySQL avançado	91
-----------------------------	-----------

Capítulo 27	– Criando e gerenciando permissões de acesso TCP/IP.....	92
Capítulo 28	– Criando um novo usuário.....	95
Capítulo 29	– Excluindo um usuário	99
Capítulo 30	– Modificando senhas de usuários	101
Capítulo 31	– Criando <i>stored procedures</i>	102
Capítulo 32	– Criando <i>triggers</i> (gatilhos)	106
Capítulo 33	– Criando um script de banco de dados compatível com o MySQL	110
Capítulo 34	– Manutenção do banco de dados	114
Capítulo 35	– Fazendo backup dos dados	115
Capítulo 36	– A função de um DBA (<i>DataBase Administrator</i>)	117

Anexo

Funções em MySQL	91
-------------------------------	-----------

Parte 1

Introdução

Capítulo 1

O que este livro aborda?

Este livro foi escrito para profissionais que desejam utilizar o banco de dados MySQL nas suas aplicações desktop e Web. É uma obra interessante, pois não exige muito conhecimento técnico por parte do(a) leitor(a); o autor apresenta solução em MySQL do básico ao avançado, permitindo, assim, que os assuntos aqui apresentados sejam facilmente assimilados.

Na primeira parte do livro, serão abordados assuntos gerais relacionados a bancos de dados e SGBD's, nos quais o(a) leitor(a) terá a oportunidade de conhecer o MER (Modelo de Entidades e Relacionamentos), além de construir um modelo de banco de dados completo utilizando-o.

A segunda parte é dedicada à instalação e configuração do MySQL. Será feita uma apresentação completa dos tipos de dados suportados por este SGBD. Também será realizada a instalação e apresentação do HeidiSQL, um gerenciador gráfico do MySQL, utilizado no livro para realizar a criação e o gerenciamento do banco de dados.

A utilização prática no MySQL começa na terceira parte, mas, antes disso, será feita uma revisão completa da linguagem SQL e de seus comandos `CREATE`, `DROP`, `ALTER`, `INSERT`, `DELETE`, `SELECT` e `UPDATE`, além das funções predefinidas da SQL. Aqui serão realizadas, de forma fácil e objetiva, as operações de ativar banco de dados, excluir banco de dados, criar, excluir e modificar tabelas, bloquear e desbloquear tabelas, visualizar propriedades de tabelas, inserir, visualizar e excluir dados nas tabelas, além de executar consultas simples e complexas de dados. É interessante observar que, nesta parte, será utilizado o MER criado na primeira matéria do livro, permitindo, assim, que o(a) leitor(a) aprenda a utilizar o MER na criação de seus bancos de dados.

A quarta e última parte do livro é dedicada a assuntos avançados, como a criação de permissões de acesso aos bancos de dados do MySQL via TCP/IP, dicas de segurança nos dados, funções de criptografia, criação e exclusão de usuários do MySQL, redefinição de senhas de acesso, *stored procedures* e *triggers*, scripts, manutenção e backups do banco de dados. Para finalizar, teremos uma visão do autor sobre a função de um DBA (*Data Base Administrator*) no atual ambiente de desenvolvimento de software.

Capítulo 2

Bancos de dados

Um banco de dados é um conjunto de dados armazenados em um computador. Esses dados, se observados separadamente, não têm valor nenhum, mas, quando utilizados em ordem, revelam informações que poderão ser usadas futuramente, por isso, um banco de dados deve ser seguro e nunca ficar exposto a pessoas não autorizadas.

Os sistemas comerciais normalmente precisam de um banco de dados para funcionar. Na realidade, eles trabalham para armazenar dados e recuperá-los de um banco de dados. Quando adquirido um destes sistemas no mercado, mesmo que sejam sistemas proprietários, isso não quer dizer que a empresa proprietária do software seja dona do banco de dados de seus clientes, pois os dados são de propriedade exclusiva da pessoa a que se referem, e não de quem oferece recursos de software para manter o banco de dados!

Existem vários bancos de dados no mercado, mas é importante escolher bem este software, antes de começar a desenvolver o seu sistema. O MySQL é um banco de dados robusto, que compete com os grandes do mercado. A vantagem do MySQL está nas suas licenças: uma delas permite que ele seja utilizado gratuitamente em sua empresa, uma segunda, que deve ser paga, permite que o MySQL seja distribuído junto com outros softwares para seus clientes, permitindo que eles tenham um banco de dados robusto sem que precisem pagar nada por isso.

Vamos às características comuns aos atuais bancos de dados:

- **menor redundância:** bancos de dados permitem que as informações de uma entidade (cliente, estoque, caixa etc.) sejam relacionadas com informações de outras entidades, permitindo o compartilhamento entre elas. Esta possibilidade de compartilhar dados permite uma opção de consulta de dados e geração de informações que se referem a várias entidades ao mesmo tempo, sem redundância;
- **segurança:** um Sistema de Gerenciamento de Banco de Dados, que será visto mais adiante, só permitirá o acesso aos dados após um log-in de usuário, ou seja, mediante log-in e senha definidos pelo administrador do sistema. Isto inibe o acesso de curiosos e pessoas não autorizadas;

- **integridade:** bancos de dados possuem meios para não permitir operações que comprometam a integridade dos dados, como, por exemplo, a exclusão de um item de nota fiscal já impressa. Isto jamais deve ser permitido, pois afetará diretamente a normas e leis vigentes;

Observação: com o desenvolvimento em camadas, essa função foi direcionada para a camada de Negócio. Mesmo assim, abordarei as *stored procedures* e *triggers*, pois considero que ainda existam sistemas que as utilizem. Será importante entender o funcionamento delas, caso seja necessário dar manutenção em alguma destas rotinas!

- **compartilhamento:** é uma das maiores vantagens dos bancos de dados, pois permite acesso aos dados por vários usuários ao mesmo tempo.

Capítulo 3

Modelagem de dados

A modelagem de dados é uma área da TI (Tecnologia da Informação) que estuda e apresenta ferramentas para descrever os dados de forma conceitual, ou seja, utilizando símbolos e gráficos. A simbologia utilizada favorece uma representação mais precisa e rápida dos dados, permitindo que qualquer banco de dados seja projetado antes de sua real criação. Este não é o primeiro passo no desenvolvimento de um software, pois, antes da criação de um modelo de dados, a Engenharia de Software aborda a Análise de Requisitos de sistemas.

Existem vários modelos de dados, mas os mais utilizados ultimamente são: modelos orientados a objetos e, principalmente, os modelos relacionais.

Os modelos orientados a objetos representam os dados mediante objetos, ou seja, um conjunto de objetos que contém valores armazenados. Cada objeto deste modelo possui métodos que operam um dado elemento. Os objetos são agrupados em classes. Este modelo é muito complexo e não faz parte do foco deste livro, pois o MySQL é um banco de dados relacional. Utilizaremos o Modelo de Entidades e Relacionamentos para descrever o banco de dados que iremos criar e gerenciar no MySQL.

Vamos então ao Modelo de Entidades e Relacionamentos, mais conhecido por MER!

MER (Modelo de Entidades e Relacionamentos)

Este modelo representa um conjunto de tabelas que são utilizadas para armazenar os dados. Essas tabelas possuem relacionamentos que permitem o acesso ao banco de dados de uma só vez.

Os relacionamentos de um banco de dados relacional podem ser físicos ou lógicos. Quando o relacionamento é físico, ele está armazenado no próprio banco de dados, mas um relacionamento lógico pode ser definido por uma linguagem de acesso aos dados, como a SQL, abordada na terceira parte deste livro.

O Modelo de Entidades e Relacionamentos possui símbolos que representam as entidades, relacionamentos e o conjunto de atributos que vão designar o banco de dados. Vamos estudar esses símbolos:

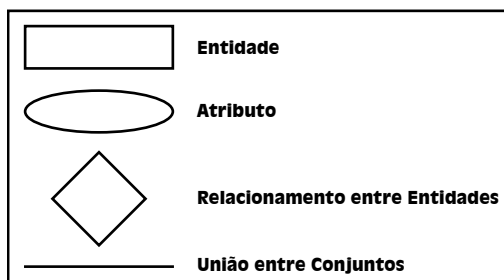


Figura 3.1: Símbolos do MER.

- **entidade:** representa um conjunto de objetos ou eventos do mundo real. Como exemplo, podemos citar um conjunto de clientes, de planilhas de faturamento, ordens de serviço, notas fiscais etc.;
- **atributo:** representa os tipos de dados armazenados. Eles descrevem as características das entidades. O termo `preco_de_venda` pode ser um atributo da entidade `Produtos`. Mais adiante, veremos um exemplo completo sobre este assunto;
- **união entre conjuntos:** faz a união entre uma ou mais entidades;
- **relacionamento entre entidades:** identifica a união entre as entidades. Tem a função de representar os atributos envolvidos no relacionamento.

Existe neste modelo outro termo, chamado de tupla. A tupla representa o conjunto de todos os atributos das entidades. Em um banco de dados, as tuplas são representadas pelas linhas das tabelas e, caso exista um relacionamento entre tabelas e colunas de outras tabelas também poderão fazer parte de uma tupla de consulta em SQL. Isto será visto na prática mais adiante.

Cardinalidade |||||

O relacionamento entre as entidades possui uma característica importante: cardinalidade. Ela representa o número de elementos das entidades envolvidas no relacionamento, ou seja, permite iden-

tificar a quantidade de tuplas em cada entidade envolvida em cada relacionamento. Veremos quais são os tipos de relacionamentos possíveis:

- **1..1 (um para um):** um elemento da entidade A pode se relacionar com apenas um elemento da entidade B:

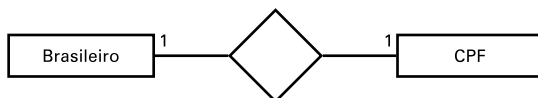


Figura 3.2: Exemplo de cardinalidade um para um. Lê-se: um brasileiro pode ter um CPF.

- **1..N (um para muitos):** um elemento da entidade A pode se relacionar com vários elementos da entidade B:

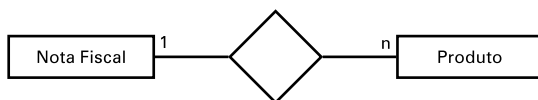


Figura 3.3: Exemplo de cardinalidade um para muitos.

Lê-se: uma nota fiscal pode ter vários produtos.

- **N..N (muitos para muitos):** vários elementos da entidade A podem se relacionar com vários elementos da entidade B:

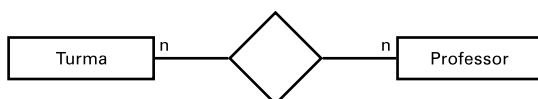


Figura 3.4: Exemplo de cardinalidade muitos para muitos.

Lê-se: uma turma pode ter vários professores e um professor pode ter várias turmas.

Modelo de Entidades e

Relacionamentos: um exemplo |||||

Cenário

Uma empresa precisa implementar um sistema que deve registrar os movimentos do caixa e das contas da empresa no banco. Os cheques da empresa a serem lançados nas contas de cada banco devem ter sido antes movimentados no caixa, por questões de segurança e para evitar lançamentos indevidos:

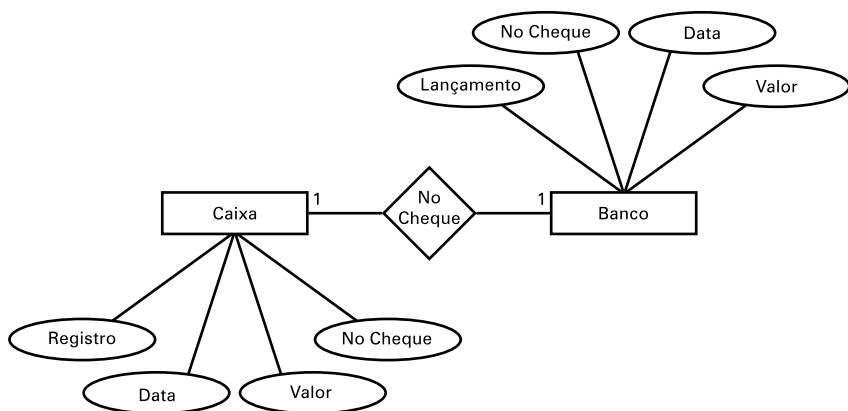


Figura 3.5: Exemplo de MER.

Solução

Observe o MER da figura anterior. O relacionamento `No _ Cheque` incluído neste modelo define que os números de cheques a serem lançados no banco deverão ter sido movimentados antes no caixa, pois, assim, não serão permitidos lançamentos indiscriminados de cheques da empresa.

Como existe um relacionamento entre as entidades `Caixa` e `Banco`, o número de cada cheque lançado na entidade `Banco` deverá estar registrado na entidade `Caixa` e não poderá ser lançado mais de uma vez em nenhuma das duas entidades, pois tem o relacionamento 1..1, o que significa que uma tupla da entidade `Banco` corresponderá a apenas uma única tupla da entidade `Caixa`.

Esta é a função do MER: definir a estrutura dos dados e permitir que regras de negócio sejam elaboradas antes da criação do banco de dados.

Capítulo 4

Criando um MER

Neste capítulo, vamos criar um MER que será utilizado mais adiante para a confecção de um banco de dados no MySQL. Como já foi visto um exemplo do MER, agora será fácil criar um Modelo de Entidades e Relacionamentos completo.

O modelo que vamos criar será para um software de controle financeiro pessoal e terá as seguintes tabelas:

- CaixaMovimento;
- HistoricoPadrao;
- ContaBanco;
- BancoMovimento;
- ContasPagarReceber.

As tabelas de nosso banco de dados devem ter os seguintes atributos:

CaixaMovimento

LancamentoCaixa
CodigoHistorico
Documento
Data
Valor

Tabela 4.1.

HistoricoPadrao

CodigoHistorico
Descricao

Tabela 4.2.

ContaBanco

Conta
Agencia
NomeBanco

Tabela 4.3.

BancoMovimento
LancamentoBanco
Conta
LancamentoCaixa
Data
TipoLancamento
Valor

Tabela 4.4.

ContasPagarReceber
LancamentoContPG
CodigoHistorico
DataVencimento
Valor
Tipo
Situacao

Tabela 4.5.

Observe que os nomes das tabelas e seus atributos não possuem espaço, caracteres especiais ou acentos, pois estes causariam problemas em sistemas de bancos de dados e no MySQL não seria diferente. Por isso, jamais deverão ser utilizados estes caracteres em um MER. Agora, veja como ficará o nosso modelo:

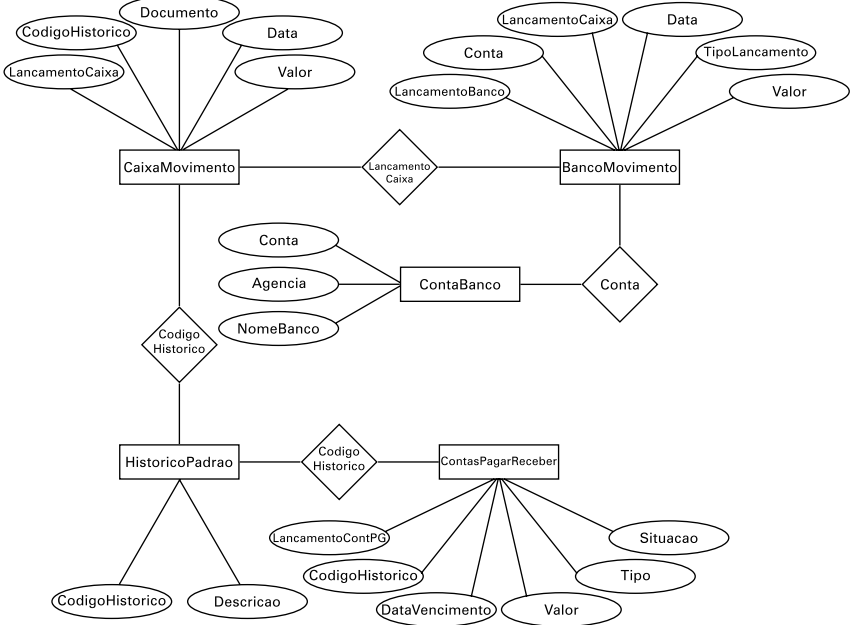


Figura 4.1: Modelo de Entidades e Relacionamentos utilizado neste livro.

O modelo da **Figura 4.1** possui quatro relacionamentos, utilizados para eliminar redundância nos dados. Observe que a tabela `HistoricoPadrao` está relacionada com outras tabelas, ou seja, suas tuplas serão úteis tanto para a tabela `CaixaMovimento` quanto para a tabela `ContasPagarReceber`. Neste modelo, o usuário do sistema lançará um histórico padrão, que é uma descrição comum do tipo: “Pagamento de contas” ou “Contas recebidas” etc., comuns tanto para `CaixaMovimento` quanto para `ContasPagarReceber`.

O relacionamento entre as tabelas `CaixaMovimento` e `BancoMovimento` refere-se a lançamento de cheques do banco e no caixa. Um lançamento de cheque no banco só será permitido após um lançamento deste mesmo cheque no caixa. Este modelo será implementado mais adiante no MySQL.

Capítulo 5

SGBD's (Sistemas de Gerenciamento de Bancos de Dados)

Nos “primórdios” da computação, os dados eram gravados em arquivos de textos. A palavra texto, por si só, demonstra o quanto a segurança destes sistemas era falha! Como sabemos, um texto pode ser lido por diversos aplicativos existentes no mercado. Por isso, teve início a preocupação em criar sistemas capazes de armazenar dados de forma mais ordenada e sistemática, apesar de não se pensar muito em segurança, apenas em limitar o acesso destes arquivos a sistemas específicos. Naquele momento, nasciam os tão conhecidos DBF's.

Com o advento dos arquivos DBF's e DB's, que vieram depois dos arquivos de texto, os dados eram gravados em documentos estruturados, capazes de armazenar informações com uma estrutura predefinida, mas ainda haviam limitações quanto ao tamanho máximo destes arquivos e limites de armazenamentos.

Os SGBD's foram, então, criados para melhorar o gerenciamento dos dados, oferecendo melhorias significativas em relação aos arquivos de armazenamento. Os SGBD's são verdadeiros sistemas que têm a função de armazenar os dados e, também, oferecer mecanismos para o gerenciamento das informações. Para que um banco de dados seja um SGBD, ele deve atender todas as seguintes características:

- exigir um log-in e senha para permitir o acesso aos dados;
- serem compatíveis com a linguagem SQL (*Structured Query Language*);
- utilizar DDL (Linguagem de Definição de Dados) e DML (Linguagem de Manipulação de Dados), grupos distintos da SQL para criar e manipular dados;
- permitir o acesso de vários usuários ao mesmo tempo;
- ser robusto, ou seja, permitir grande armazenamento de dados;
- permitir criação de *stored procedures*, ou seja, rotinas programadas internamente;
- suportar conexões TCP/IP.

Veja os SGBD's mais conhecidos atualmente no mercado:

- MySQL;
- Oracle;

- DB2;
- SQL Server;
- Interbase;
- PostgreSQL;
- Sybase.

Mais adiante, veremos como utilizar a SQL para criar e manipular os dados no MySQL, um dos SGBD's mais utilizados no mercado, e, atualmente, imbatível no ambiente Web.

Parte 2

O banco de dados MySQL

Capítulo 6

Histórico

Em meados de 1997, a Internet crescia assustadoramente e os SGBD's utilizados nas empresas eram voltados para robustez, segurança e confiabilidade. Naquele momento, surgiam duas novas características que deveriam estar em um SGBD: rapidez e flexibilidade. Naquela época, era difícil encontrar um sistema de gerenciamento de dados com todas estas características reunidas, pois as empresas precisavam, naquele momento, de mais performance e flexibilidade. Reunir todas estas características em um único sistema é realmente um desafio. Foi então que um grupo de desenvolvedores criou um sistema para armazenamento e gerenciamento de dados rápido, flexível e seguro. Nasceu o MySQL.

Por sua rapidez e flexibilidade, o MySQL popularizou-se rapidamente na Internet, pois, além de ser veloz, é um SGBD que roda em várias plataformas. Porém, não ficou apenas nisso, o MySQL tornou-se conhecido no mundo inteiro, conquistando usuários não só de servidores de Internet, mas, também, de sistemas locais e redes Ethernet.

Até a quinta versão, o MySQL foi muito utilizado na Internet, após isso, além da Internet, ele está sendo utilizado em aplicações para computadores locais, pois, a partir de sua quinta versão, ele passou a ter características compatíveis com os robustos servidores de banco de dados, como, por exemplo, tabelas seguras e transações. A partir daí, este SGBD tornou-se um forte sistema para ser utilizado tanto em máquinas locais quanto em computadores remotos e Internet, pois continua oferecendo recursos para robustez, performance e flexibilidade. Basta ao usuário escolher o formato de tabela necessário para sua aplicação, optando entre segurança nos dados e performance.

Quando este livro foi escrito, o MySQL estava na versão 5.0.51a. Este SGBD é mantido pela MySQL AB, empresa fundada pelos seus desenvolvedores. A MySQL AB tem a missão de fornecer serviços de suporte para os usuários do MySQL, mas, para isso, estes serviços devem ser contratados. No próximo capítulo, veremos a licença do MySQL.

O golfinho de nome Sakila, logo do MySQL, foi escolhido pelos fundadores da MySQL AB, mediante um concurso promovido pela própria empresa.

Atualmente, o MySQL é um software com licença GPL (*General Public License*), ou seja, pode ser utilizado e modificado para uso próprio sem qualquer custo. Trata-se do banco de dados de código aberto mais popular do mundo. Vejamos, então, como é a licença do MySQL.

Capítulo 7

Licença de uso

O MySQL está sob a licença GPL (*General Public License*). A GPL é uma licença que permite que o produto seja utilizado por qualquer pessoa. O código deste produto é aberto e pode ser modificado para uso próprio, ou seja, se você tem uma empresa, vai utilizar o MySQL e não necessita de suporte técnico, poderá utilizar o MySQL sem nenhum custo.

Um detalhe importante da GPL é o seguinte: se um produto sob a licença GPL for distribuído em conjunto com outro produto proprietário, o produto sob a GPL deve ser licenciado, isto quer dizer que, se você desenvolver um software e pretender distribuí-lo junto com o MySQL para seus clientes, deve adquirir a licença comercial do MySQL. Olhando para a relação custo/benefício, isto não é ruim, pois distribuir produtos sob licença GPL é um benefício que uma empresa trará a seus clientes, e isto pode fazer diferença em um negócio!

Caso queira desenvolver um software para utilizar o MySQL em sua própria empresa, poderá adquirir o MySQL normalmente, sem nenhum custo. Poderá até modificar o MySQL para uso próprio, pois ele tem código aberto! O site para download do MySQL será visto no próximo capítulo.

Lembre-se de que o MySQL é um banco de dados robusto, com todos os requisitos necessários para uma utilização altamente profissional. Hoje em dia, está em praticamente todos os servidores de hospedagem do mundo. Utilizar MySQL significa utilizar um SGBD profissional, flexível, seguro, robusto e com custo zero para sua empresa!

Capítulo 8

Instalando e configurando o MySQL

A partir de agora, veremos os procedimentos para instalação e configuração do MySQL. Para tanto, é preciso fazer o download do programa. Este livro foi baseado na versão 5.0.51a e baixamos no endereço: <http://dev.mysql.com/downloads/>.

Ao entrar no site, procure o download para **MySQL Community Server** e escolha a versão anotada como **Current Release (Recommended)**. Após entrar no link, escolha sua plataforma e baixe a versão.

Observação: para este livro foi realizado o download **Windows ZIP/Setup.EXE (x86)** e este sistema operacional será considerado para instalação e configuração do MySQL. Porém, você pode realizar o download para sua plataforma normalmente, pois o MySQL é muito flexível e roda em vários sistemas. Para realizar o download, será necessário criar um cadastro no site, com alguns dados pessoais a serem informados. Será utilizado um e-mail e uma senha definida pelo próprio usuário para acessar a seção de downloads. Não se preocupe, esse cadastro não custa nada.

Após baixar o MySQL, vamos iniciar a instalação. A **Figura 8.1** mostra a tela inicial do instalador. Clique em **Next** para iniciar a instalação do MySQL:

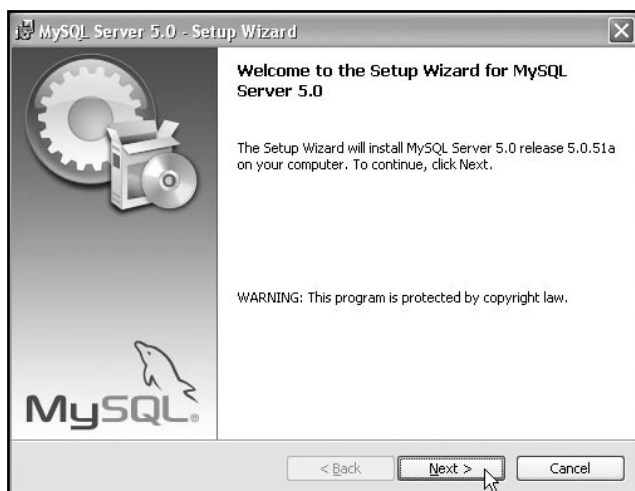


Figura 8.1: Janela inicial do instalador do MySQL.

A **Figura 8.2** define o tipo de instalação. Selecione a instalação típica, conforme a figura, depois clique em **Next >** para passar para a próxima etapa:

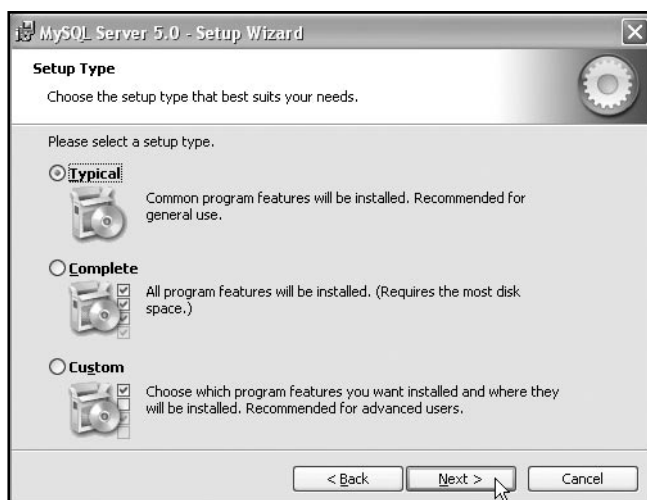


Figura 8.2: Selecionando o tipo de instalação.

A figura seguinte mostra as configurações iniciais de instalação. Clique em **Install** para iniciar a instalação do MySQL:

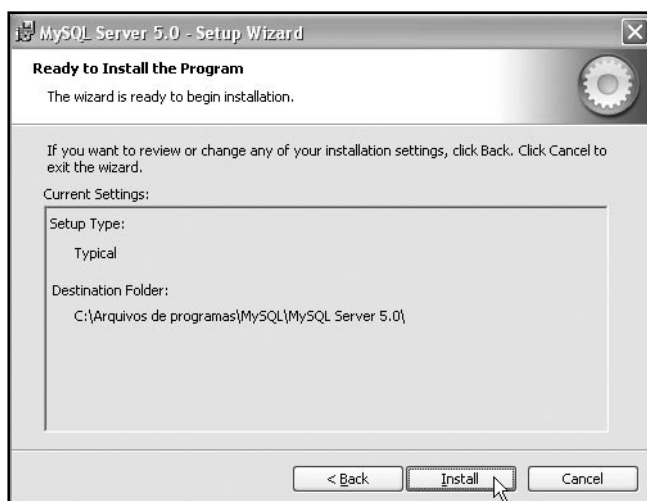


Figura 8.3: Instalando o MySQL.

Veja a janela de progresso da instalação na figura a seguir:



Figura 8.4: Progresso na instalação do MySQL.

Após instalar o pacote MySQL, o gerenciador de instalação mostrará duas janelas de apresentação do MySQL Enterprise, a versão comercial do MySQL. O botão **More...** nos dá maiores informações sobre esta versão. Clique em **Next >**, conforme as **Figuras 8.5 e 8.6**:

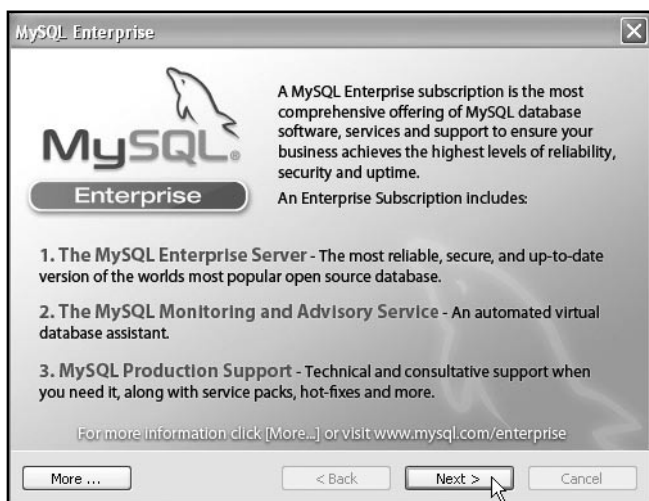


Figura 8.5: Primeira janela de apresentação do MySQL Enterprise.

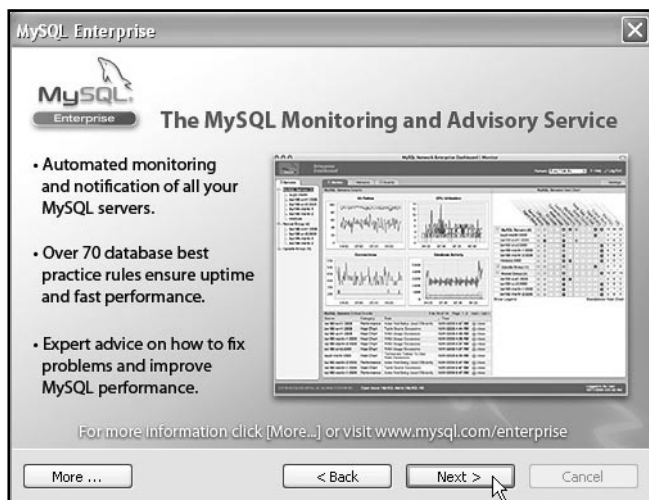


Figura 8.6: Segunda janela de apresentação do MySQL Enterprise.

Deixe marcada a opção **Configure the MySQL Server now**, conforme a figura seguinte, e utilize o botão **Finish**, para iniciar a configuração do servidor:

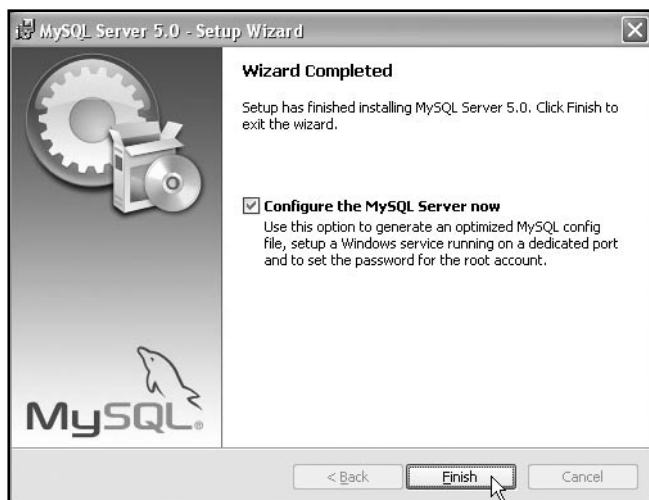


Figura 8.7: Finalizando a instalação e iniciando a configuração do MySQL.

Será apresentada a janela inicial do gerenciador de configuração:



Figura 8.8: Janela inicial do gerenciador de configuração do MySQL.

Definindo a configuração do servidor MySQL

O gerenciador de configuração do MySQL permite dois tipos de configuração:

- configuração detalhada;
- configuração simples.

Será feita a configuração detalhada (**Detailed Configuration**) por esta ser a mais completa. Ela permite a definição de várias características de serviço, suporte e funcionalidades do servidor MySQL. Marque a opção **Detailed Configuration** e clique em **Next >**, conforme a Figura 8.9:

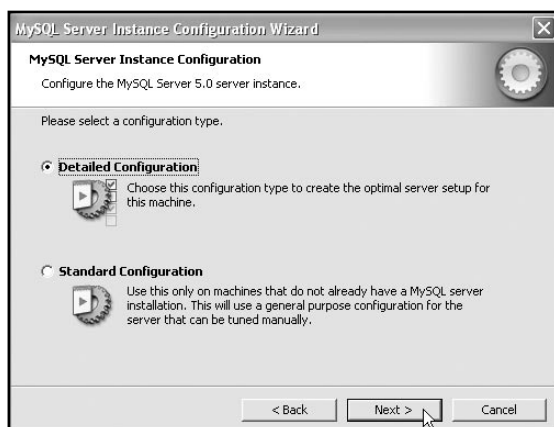


Figura 8.9: Definindo o tipo de configuração.

Definindo o ambiente de instalação

O MySQL pode ser utilizado em três tipos de ambientes:

- **Máquina de Desenvolvimento (Developer Machine):** o MySQL será configurado para utilizar menos memória, por ser instalado em um computador que possui funções que estão além do serviço de banco de dados;
- **Máquina Servidora (Server Machine):** uso médio de memória do computador, que fornecerá serviço de servidor, mas não dedicado;
- **Servidor MySQL em Máquina Dedicada (Dedicated MySQL Server Machine):** máximo uso de memória. É recomendado apenas para instalação do MySQL em computadores que irão oferecer serviços exclusivos do banco de dados MySQL.

Neste livro será utilizada a primeira opção, **Developer Machine**. Esta opção deve ser marcada caso o MySQL seja instalado em um computador que precise executar outras tarefas:



Figura 8.10: Definindo o ambiente de instalação.

Definindo o tipo de tabela a ser usada no banco de dados

O banco de dados MySQL possui vários formatos de tabelas básicos, que são os seguintes:

- ISAM;

- HEAP;
- MyISAM;
- InnoDB;
- BDB.

As tabelas BDB e InnoDB são seguras, pois aceitam transação. Recomenda-se o uso destas tabelas, pois este formato confere maior segurança nos dados e permite que falhas no equipamento ou no servidor não causem perda total. Utilizaremos a primeira opção do gerenciador de configuração, **Multifunctional Database**, que permite criar tabelas nos formatos InnoDB ou MyISAM apenas. Esta opção é interessante por nos permitir o uso de tabelas do tipo segura (InnoDB) ou rápida (MyISAM). Utilize sempre o formato InnoDB, deixando MyISAM apenas para casos especiais, onde seja necessária performance, e não segurança nos dados. Veja a **Figura 8.11**:

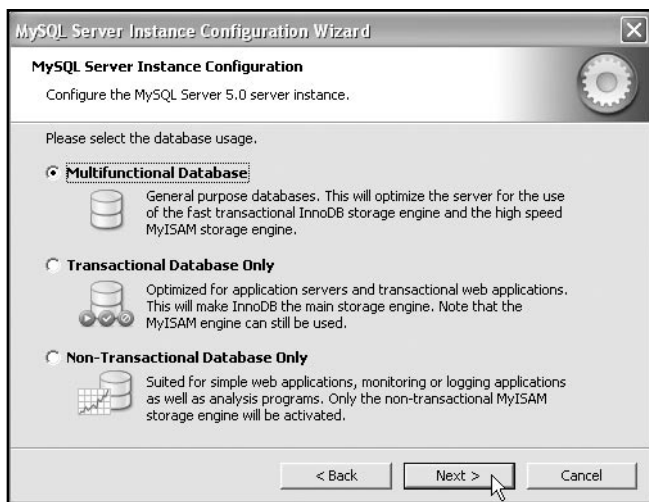


Figura 8.11: Definindo o tipo de tabela a ser usada no MySQL.

Definindo Space InnoDB

A próxima janela será para definição do *Space* das tabelas InnoDB, ou seja, a pasta em que estas tabelas serão armazenadas. Escolha a partição e a pasta de sua preferência. Veja a figura seguinte, na qual é selecionada a unidade *D:* e a pasta `|MySQL Datafiles|` para guardar as tabelas InnoDB, criadas no MySQL.

Mais adiante veremos como criar tabelas InnoDB no MySQL:

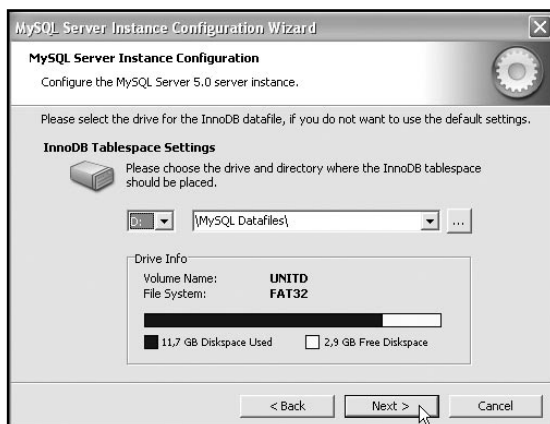


Figura 8.12: Definindo o local das tabelas InnoDB.

Conexões concorrentes

O MySQL permite definir o número de conexões concorrentes. Este limite define o número máximo de usuários logados no banco de dados. Isto varia muito de um caso para outro. Em um banco de dados Web, o número de conexões concorrentes deve estar na casa dos 500 ou mais, já em um sistema de rede que necessita de muita segurança de acesso, e possui usuários bem definidos, este número de conexões concorrentes deve ser o necessário.

Será utilizada uma definição manual, conforme a **Figura 8.13**, mas veja qual é a real necessidade de sua empresa para definir este limite de usuários logados no MySQL:

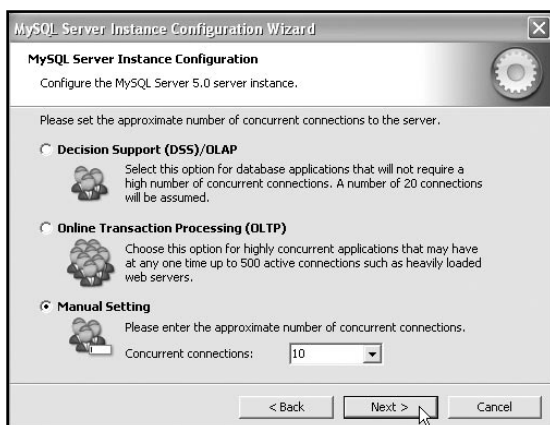


Figura 8.13: Definindo o número de conexões concorrentes no MySQL.

Habilitando conexões TCP/IP para o MySQL

O MySQL utiliza a porta 3306 como padrão para conexão TCP/IP. Esta porta pode ser modificada pela opção **Port Number**. Veja a **Figura 8.14**. Habilite a opção **Enable TCP/IP Networking** para que o banco de dados aceite conexões remotas via TCP/IP. Caso esta opção não seja habilitada, o MySQL somente aceitará conexões locais.

Firewall é um software ou hardware de monitoramento que controla o tráfego nas portas TCP/IP. Mantenha um firewall ativo na rede para melhorar a segurança! A opção **Add firewall exception for this port** permite adicionar exceções do firewall para a porta TCP/IP usada pelo MySQL:

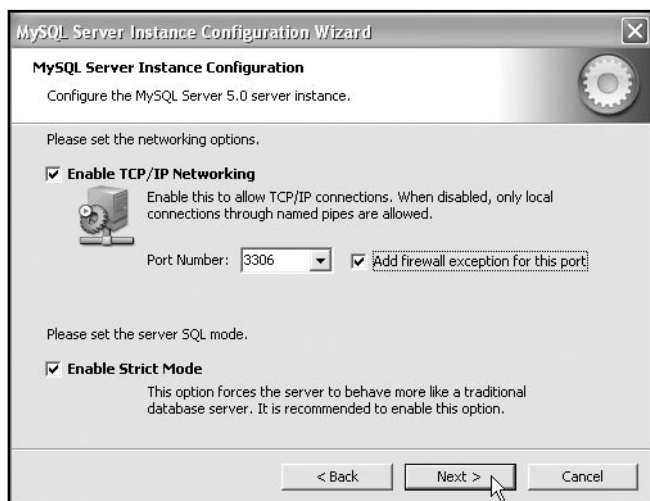


Figura 8.14: Habilitando conexões TCP/IP para o MySQL.

Suporte de caracteres

O MySQL permite definir suporte a vários padrões de caracteres. Utilize a opção **Best Support For Multilingualism** para que o MySQL defina o UTF8 como padrão.

O UTF8 é o padrão recomendado, pois ele oferece suporte para o armazenamento de dados em diferentes linguagens. Veja a **Figura 8.15**:

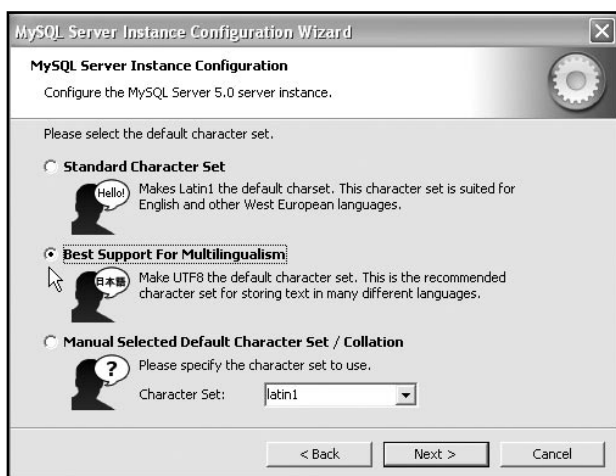


Figura 8.15: Definindo suporte de caracteres para múltiplas linguagens.

Instalando o serviço

Recomenda-se que o serviço do MySQL seja instalado no sistema e iniciado automaticamente. Isto garantirá que o servidor esteja sempre ativo, permitindo, assim, acesso a seus usuários.

Marque a opção **Install As Windows Service** para instalar o serviço. Marque a opção **Launch the MySQL Server automatically** para iniciar o servidor automaticamente. Veja esta configuração na **Figura 8.16**:



Figura 8.16: Instalando o serviço do MySQL e definindo sua execução automática no sistema.

Configurando nova senha do root

O root é o login do administrador do MySQL. Sua senha deve ser redefinida para garantir segurança no banco de dados. Recomenda-se o uso de senhas que combinam caracteres alfanuméricos e especiais; isto dificultará possíveis tentativas de quebra de segurança no sistema.

Será definida aqui a senha *senh%01*. Esta senha será considerada durante todo o conteúdo deste livro, mas você deve utilizar outra, que seja de sua confiança. Lembre-se de usá-la, neste livro, sempre que for mencionada a senha *senh%01*.

Para uma maior segurança no banco de dados, a opção **Enable root access from remote machines** não será marcada. Isto irá bloquear o acesso do usuário *root* ao banco de dados a partir de máquinas remotas. Veja a figura seguinte:



Figura 8.17: Definindo nova senha para o usuário *root*.

Executando o serviço no sistema

Neste momento, o serviço do servidor MySQL já estará pronto para ser executado.

Conforme a **Figura 8.18**, clique em **Execute** para iniciar o serviço:

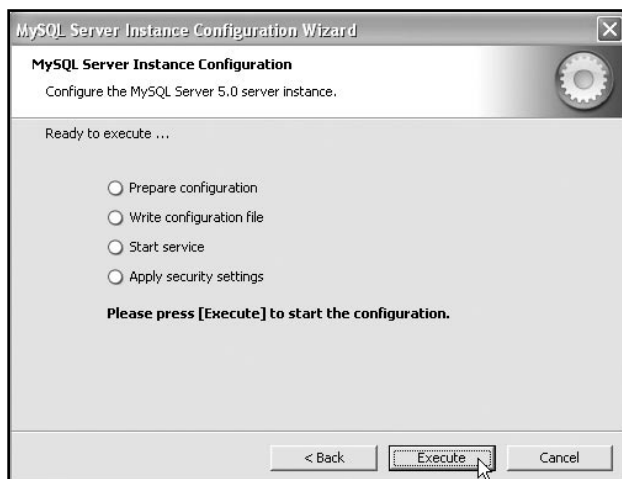


Figura 8.18: Executando o serviço no sistema.

O servidor será iniciado. Se tudo ocorreu bem, o serviço de banco de dados do servidor MySQL será executado conforme a **Figura 8.19**:



Figura 8.19: Finalizando a configuração do MySQL.

Estando todos os itens marcados, conforme a figura anterior, o servidor já estará pronto para ser utilizado.

No próximo capítulo, apresentaremos os tipos de dados suportados pelo MySQL. No **Capítulo 10**, será visto o HeidiSQL, um dos gerenciadores do MySQL *open source* mais populares do mercado. Ele será utilizado para criar e gerenciar nosso banco de dados.

Capítulo 9

Tipos de dados do MySQL

Quando criamos uma tabela em um banco de dados, é necessário definir os tipos de dados armazenados em cada coluna da tabela. As colunas de uma tabela em um SGBD representam os atributos das entidades do MER; analogamente, uma tabela em um SGBD representa uma entidade em um MER.

A tabela a seguir apresenta os tipos de dados do MySQL, seus respectivos limites de armazenamento e para que serve cada um deles.

Observação: será utilizado M para representar o tamanho máximo do display e D para o número de dígitos após o ponto decimal.

Coluna	Armazenamento
TINYINT[(M)]	Inteiros na faixa de -127 até 128.
BIT	Qualquer um destes três campos pode armazenar 0 ou 1.
BOOL	
BOOLEAN	
SMALLINT[(M)]	Inteiros de -32768 até 32767.
MEDIUMINT[(M)]	Inteiros de -8388608 a 8388607.
INT[(M)]	Inteiros de -2147483648 a 2147483647.
INTEGER[(M)]	Sinônimo de INT.
BIGINT[(M)]	Inteiros de -9223372036854775808 até 9223372036854775807.
FLOAT(D)	Número de ponto flutuante com até 24 casas decimais.
FLOAT[(M,D)]	Número de ponto flutuante com valores que estão entre -3.402823466E+38 e -1.175494351E-38, 0 e entre 1.175494351E-38 e 3.402823466E+38.
DOUBLE[(M,D)]	Número de ponto flutuante com valores que estão entre -1.7976931348623157E+308 e -2.2250738585072014E-308, 0 e entre 2.2250738585072014E-308 e 1.7976931348623157E+308.
REAL[(M,D)]	Sinônimo de DOUBLE.
DATE	Uma data entre '1000-01-01' e '9999-12-31'.

DATETIME	Combinação de data e hora na faixa entre '1000-01-01 00:00:00' e '9999-12-31 23:59:59'.
TIMESTAMP[(M)]	Um <i>timestamp</i> entre '1970-01-01 00:00:00' e algum momento no ano 2037.
TIME	Uma hora entre '-838:59:59' e '838:59:59'.
YEAR[(2 4)]	Ano de dois ou quatro dígitos com valores que podem estar entre 1901 e 2155, 0000 no formato de quatro dígitos, e 1970-2069 se estiver usando o formato de 2 dígitos (70-69).
CHAR(M)	Uma <i>string</i> de tamanho fixo na faixa de 1 a 255 caracteres.
VARCHAR(M)	Uma <i>string</i> de tamanho variável na faixa de 1 a 255 caracteres.
BLOB TEXT	Qualquer um destes campos tem tamanho máximo de 65535 ($2^{16} - 1$) caracteres. A diferença entre eles é que o BLOB é <i>case-sensitive</i> , ou seja, diferencia caracteres maiúsculos de minúsculos.
MEDIUMBLOB MEDIUMTEXT	Qualquer um destes campos tem tamanho máximo de 16777215 ($2^{24} - 1$) caracteres.
LONGBLOB LONGTEXT	Qualquer um destes campos tem tamanho máximo de 4294967295 ou 4G ($2^{32} - 1$) caracteres.

Tabela 9.1.

Quando criamos uma tabela em um banco de dados, devemos observar o tipo de dado adequado para ser utilizado em cada coluna da tabela. Imagine que seja criada uma coluna para armazenar a idade de uma pessoa. Não seria necessário utilizar o tipo `INTEGER`, pois um dado do tipo `TINYINT` resolveria o problema, pelo menos eu não conheço nenhum caso especial em que o `TINYINT` não fosse o suficiente para armazenar a idade de uma pessoa!

O problema de utilizar tipos de dados com capacidades muito além do que é necessário está no grande volume de dados gerados desnecessariamente. Lembre-se de não criar colunas nas tabelas com limites que irão esgotar nos próximos dez anos, mas utilize o tipo de dado necessário para armazenar os valores com uma pequena folga; isto, a princípio, não fará muita diferença, mas quando as tabelas do banco de dados ficarem robustas, irão influenciar bastante o desempenho do SGBD!

Capítulo 10

HeidiSQL: ferramenta gráfica *open-source* de gerenciamento do MySQL

O HeidiSQL é um dos gerenciadores do MySQL mais populares do mercado. É um software intuitivo e fácil de usar, pois utiliza janelas e objetos gráficos em sua interface. Baixe o HeidiSQL para sua plataforma no seguinte endereço <http://www.heidisql.com>.

Instalando o HeidiSQL

Instalar o HeidiSQL é muito simples. A janela inicial do instalador será conforme a **Figura 10.1**. Clique em **Next >** para ir à próxima etapa do instalador:



Figura 10.1: Janela de boas-vindas do instalador do HeidiSQL.

Conforme a figura seguinte, leia o termo de uso do HeidiSQL e, se for o caso, marque a opção **I accept the agreement** e clique em **Next >**:

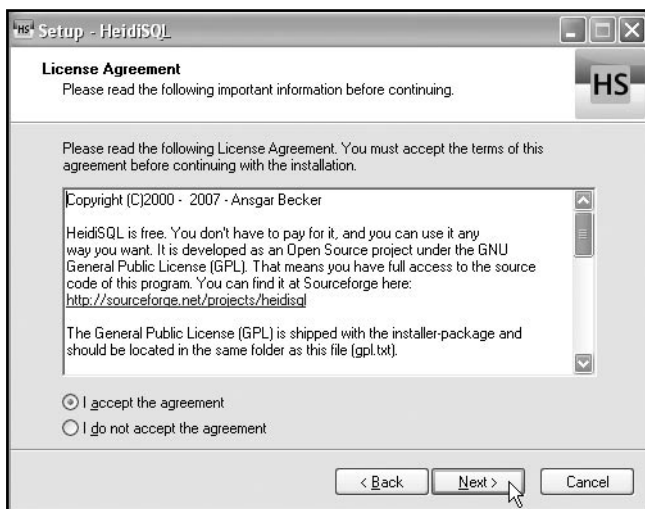


Figura 10.2: Termos de licença do HeidiSQL.

Escolha a pasta de instalação e clique em **Next >**. Veja **Figura 10.3**:

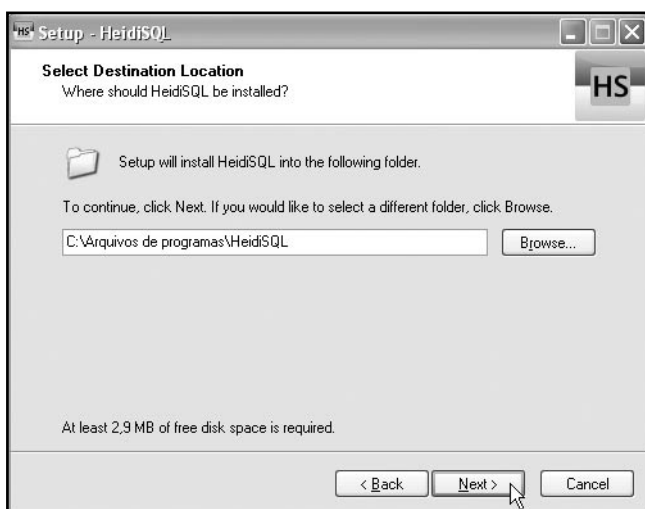


Figura 10.3: Escolhendo pasta de instalação do HeidiSQL.

Conforme a figura seguinte, o nome de menu do HeidiSQL será definido:



Figura 10.4: Definindo o nome do menu do HeidiSQL.

Marque os itens conforme a **Figura 10.5**. Isto garante a criação dos ícones e a vinculação dos arquivos de script com o HeidiSQL:

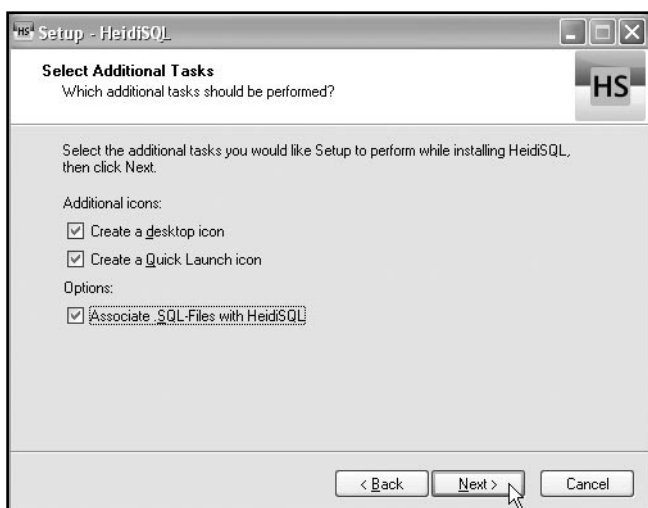


Figura 10.5: Definindo ícones do HeidiSQL.

A figura seguinte mostra as configurações antes da instalação. Clique em **Install** para instalar o HeidiSQL. Marque a opção **Launch HeidiSQL** e clique em **Finish**, conforme a **Figura 10.7**, para iniciar o software:

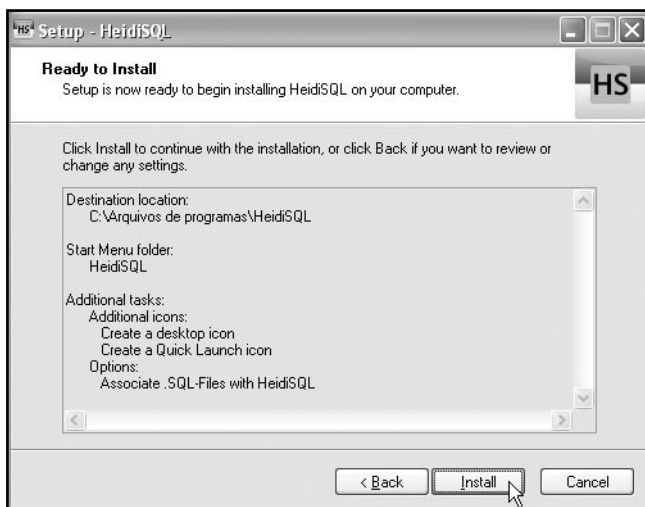


Figura 10.6: Instalando o HeidiSQL.



Figura 10.7: Finalizando a instalação e executando o HeidiSQL.

Criando uma conexão com o servidor

Quando iniciar o HeidiSQL pela primeira vez, você verá uma janela conforme a figura seguinte. Clique no botão **New** para criar uma nova conexão com o banco de dados. Aparecerá uma janela como a da **Figura 10.9**. Defina o nome da conexão como *localhost*:

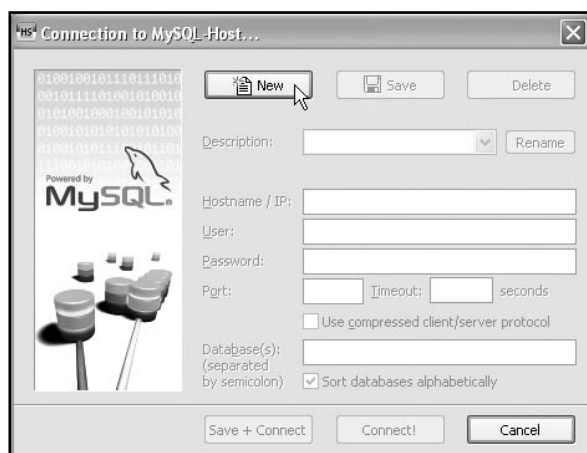


Figura 10.8: Criando uma conexão com o servidor.

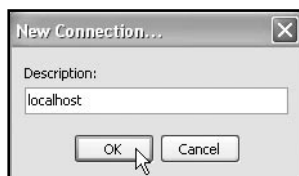


Figura 10.9: Definindo o nome da conexão local.

Clique em **OK** e defina os dados da conexão, como na **Figura 10.10**, lembrando de utilizar a senha definida na instalação do MySQL. Clique em **Save + Connect**, para salvar a conexão:

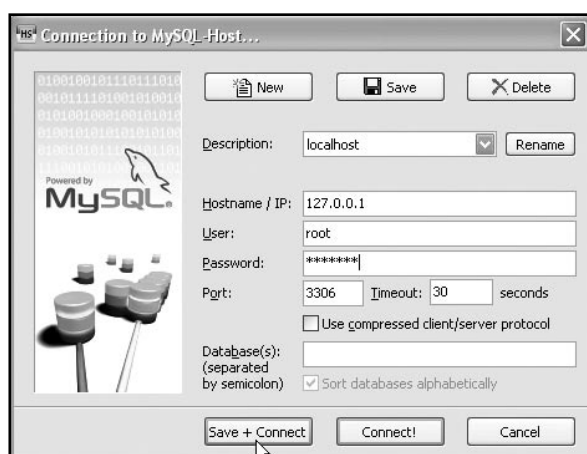


Figura 10.10: Definindo e salvando a conexão com o MySQL.

A interface do HeidiSQL

A interface do HeidiSQL é dividida em várias janelas. Veja a **Figura 10.11**:

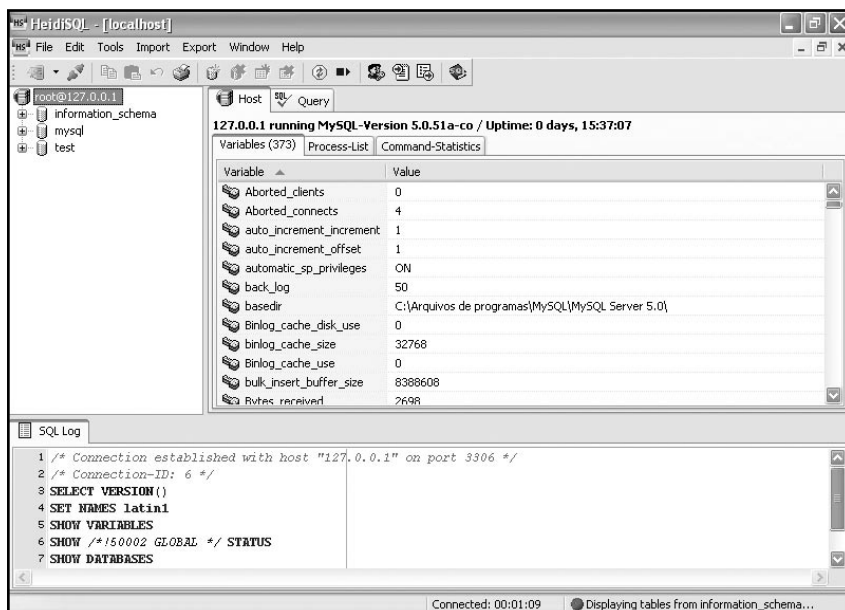


Figura 10.11: Interface do HeidiSQL.

Cada janela de interface do HeidiSQL possui a sua função. Veja as **Figuras de 10.12 a 10.17**, depois a descrição de cada uma destas janelas ou ferramentas:



Figura 10.12: Menu do HeidiSQL.



Figura 10.13: Barra de ferramentas do HeidiSQL.



Figura 10.14: Visualização do log.



Figura 10.15: Editor de campos.

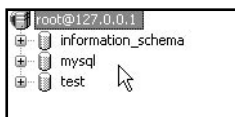


Figura 10.16: Informações de conexões e bancos de dados.



Figura 10.17: Query Manager.

Janela	Função
Menu	Contém comandos e ferramentas do HeidiSQL.
Barra de ferramentas	Atalhos para ferramentas comuns.
Visualização do Log	Exibe resultado de comandos.
Editor de campos do tipo BLOB	Permite editar campos do tipo BLOB, com uma interface mais amigável.
Informações de conexões e bancos de dados	Contém as conexões ativas e seus respectivos bancos de dados.
Query Manager	Esta é a janela mais utilizada do HeidiSQL. É nela que serão executados os comandos SQL e os comandos de gerenciamento do MySQL.

Tabela 10.1.

A partir do **Capítulo 12**, veremos como utilizar essas ferramentas na prática. Os comandos que utilizaremos no HeidiSQL serão os comandos dos menus, barra de ferramentas ou os executados diretamente na Query Manager. Será utilizado o caractere > para fazer referência aos vários níveis de menus do HeidiSQL, como, por exemplo, para fazer a conexão com o banco de dados, será utilizado o menu **File > Connection**.

Vamos, então, à revisão da SQL, para, depois, iniciar nossos exemplos.

Parte 3

MySQL básico

Capítulo 11

Revisando SQL

A linguagem SQL (*Structured Query Language*) é uma linguagem estruturada utilizada nos SGBD's. Sua função é permitir a criação e manipulação de bancos de dados e suas respectivas tabelas. Por meio desta linguagem, podemos incluir, excluir e alterar dados em um SGBD.

A SQL é uma linguagem muito completa e pode ser utilizada da forma simples ou da forma complexa, mas o importante é que podemos criar e manipular os dados de forma rápida, direta e objetiva.

Aqui serão apresentados os comandos da SQL e suas principais utilizações. A seguir, veremos quais são os grupos de comandos SQL.

DDL e DML

A SQL possui dois grupos de comandos. Veja quais são:

- **DDL (*Data Definition Language*)**: são os comandos utilizados para criar e apagar objetos no banco de dados. Veja quais são as classes de comandos DDL:

- `_ CREATE;`
- `_ ALTER;`
- `_ DROP.`

- **DML (*Data Manipulation Language*)**: comandos utilizados para manipular os dados, ou seja, interagir com os dados armazenados no banco de dados. Os grupos são:

- `SELECT;`
- `INSERT;`
- `UPDATE;`
- `DELETE.`

Agora, veremos a sintaxe de cada comando SQL. A prática será abordada nos próximos capítulos! Não serão abordados, agora, exemplos dos comandos, apenas a sintaxe, pois estes comandos serão utilizados nos capítulos posteriores, já com o HeidiSQL para a execução.

Observação: os caracteres `<>` e `[]` não fazem parte dos comandos SQL. Os caracteres `<>` serão utilizados apenas para indicar que o conteúdo entre eles deve ser definido pelo usuário. Na

sintaxe dos comandos também aparecerão os caracteres [], e isso significa que o conteúdo entre eles é opcional. O caractere | é utilizado apenas para indicar que será válida qualquer uma das opções separadas por ele. O sinal ... não faz parte de nenhum comando, e será incluído apenas para indicar continuidade na sequência de algumas cláusulas. Cada comando deve ser finalizado por um ponto-e-vírgula (;).

Vejamos então quais são os comandos SQL utilizados para definir e manipular os dados no MySQL.

CREATE DATABASE |||

O comando CREATE DATABASE é utilizado para criar um banco de dados em um SGBD. Sua sintaxe é:

```
CREATE DATABASE <nome do banco de dados>;
```

DROP DATABASE |||

Da mesma forma que um banco de dados pode ser criado utilizando SQL, ele pode também ser excluído definitivamente do SGBD. A sintaxe do comando é a mesma do comando anterior:

```
DROP DATABASE <nome do banco de dados>;
```

CREATE TABLE |||

Este comando cria uma tabela no banco de dados. Quando criamos uma tabela em um banco de dados, devemos definir o tipo de dado para cada coluna. Veja como:

```
CREATE TABLE <Nome da tabela>
(
    <Nome da coluna1> <Tipo>[(M)] [primary key]
    [not null] [default <valor>] [auto_increment],
    <Nome da coluna2> <Tipo>[(M)] [not null]
    [default <valor>] [auto_increment]...
)
```

```
Type = InnoDB | MyISAM
;
```

Utilize a vírgula para separar as várias colunas criadas. Veja para que servem algumas cláusulas deste comando:

- `<Tipo>[(M)]`: define o tipo e o tamanho máximo de armazenamento para a coluna da tabela;
- `[primary key]`: cria uma chave primária. *Primary key* é uma chave única da tabela, ou seja, uma coluna que não poderá ter tuplas (linhas da tabela) com valores duplicados, nulos ou em branco. Cada tabela poderá ter apenas uma *primary key*;
- `[not null]`: não permite valores nulos para os dados da coluna;
- `[default <valor>]`: quando incluímos uma nova tupla (linha) em uma tabela, o valor inicial de cada coluna será NULL, até que um novo valor seja definido por um comando INSERT ou UPDATE. A cláusula `[default <valor>]` muda o valor inicial do campo, que é NULL, para o que foi definido pela cláusula, como `integer(5) default 0`. Neste caso, quando incluída uma nova tupla na tabela, mesmo que o valor da coluna não seja definido, o valor dela será zero em vez de NULL;
- `[auto_increment]`: permite uma auto-incrementação de um valor numérico. Quando for incluído um dado em uma coluna definida como `auto_increment`, esta coluna terá seu valor incluído automaticamente pelo sistema, por isso os valores deste tipo de coluna nunca deverão ser incluídos pelo usuário mediante os comandos INSERT e UPDATE.

```
ALTER TABLE |||||
```

Operações de alteração da estrutura das tabelas de um banco de dados podem ocorrer, como a inclusão de novas colunas em uma tabela, exclusão de novas colunas ou modificação de colunas existentes. É natural que uma coluna seja adicionada, removida ou modificada em uma tabela do banco de dados, após este já estar sendo utilizado pelo usuário.

Para modificar a estrutura de uma tabela, veja as opções do ALTER TABLE para cada caso:

Adicionando colunas

```
ALTER TABLE <nome da tabela>
```

```

Add column <Nome da coluna1> <Tipo>[(M)]
    [not null] [default <valor>] [auto_increment],
Add column <Nome da coluna2> <Tipo>[(M)]
    [not null] [default <valor>] [auto_increment]...
;

```

Removendo colunas

```

ALTER TABLE <nome da tabela>
    Drop column <Nome da coluna1>,
    Drop column <Nome da coluna2>...
;

```

Modificando colunas

```

ALTER TABLE <nome da tabela>
    Modify column <Nome da coluna1> <Tipo>[(M)]
        [not null] [default <valor>] [auto_increment],
    Modify column <Nome da coluna2> <Tipo>[(M)]
        [not null] [default <valor>] [auto_increment]...
;

```

DROP TABLE |||

Uma tabela pode ser excluída definitivamente do banco de dados por meio deste simples comando. Veja como:

```
DROP TABLE <nome da tabela>;
```

Para que este comando funcione corretamente, o banco de dados da tabela deve estar em uso. Isto será visto mais adiante.

INSERT |||

O comando `INSERT` é muito direto na inclusão de dados em uma tabela, é o comando que utilizaremos para incluir novas tuplas. Veja como é fácil utilizar o `INSERT`.

```

INSERT
    into <nome da tabela>

```

```

        (
            <Coluna1>,
            <Coluna2>...
        )
    Values
        (
            <Valor da coluna 1>,
            <Valor da coluna 2>...
        )
;

```

Neste comando, incluímos primeiro uma lista de nomes das colunas da tabela, separados por vírgula. Depois, na cláusula `Values`, colocamos os valores das respectivas colunas, na mesma seqüência utilizada nos nomes de colunas.

Para incluir valores em uma tabela, não precisamos utilizar todas as colunas, apenas aquelas nas quais queremos persistir os dados. Devemos prestar atenção para não definir valores para colunas como `auto_increment`, e não esquecer de definir valores para as colunas que não aceitam valores nulos, ou seja, as definidas como `not null`.

DELETE ||

Apaga tuplas em uma tabela. Este comando tem uma cláusula chamada `where` que é muito importante, pois ela define a condição para que os dados sejam apagados corretamente. Cuidado, pois a não utilização da cláusula `where` neste comando significa excluir todas as tuplas definitivamente da tabela. Sua sintaxe é a seguinte:

```

DELETE from <Nome da tabela> where <Condição de deleção>;

```

A condição de deleção deve ser uma comparação com os valores das colunas da tabela. Os operadores utilizados para a construção de condições serão vistos no **Capítulo 19**. Exemplos deste comando serão vistos mais adiante!

SELECT ||

Este é um dos comandos SQL mais utilizados em um sistema de banco de dados. Ele é utilizado para consultar os dados em uma ou

mais tabelas. É muito flexível, pois permite a união de valores de tabelas para efetuar um relacionamento lógico. Sua sintaxe é a seguinte:

```
SELECT
    <Apelido da tabela>.<Coluna da tabela> AS <apelido coluna>,
    <Apelido da tabela>.<Coluna da tabela> AS <apelido coluna>...
from
    <Tabela1> <Apelido da tabela1>,
    <Tabela2> <Apelido da tabela2>...
where <Condição da consulta>
[Group by <Coluna>]
[Order by <Coluna> ASC | DESC]
;
```

Vamos ver as cláusulas deste comando:

- **Apelido da tabela:** quando criamos um comando `SELECT`, podemos utilizar várias tabelas na consulta. Podemos, então, definir uma letra como apelido de uma tabela e, quando for necessário utilizar o nome da tabela no comando, utilizaremos seu apelido, que pode ser um nome muito menor e mais fácil de trabalhar;
- **Coluna da tabela:** representa uma coluna de uma tabela;
- **Apelido coluna:** um apelido para a coluna. Este apelido aparecerá no resultado da consulta, no lugar do nome original da coluna;
- **From:** cláusula que define quais tabelas serão utilizadas na pesquisa;
- **Where:** cláusula que define uma condição (filtro) para a consulta. Esta condição filtrará e permitirá a execução do comando apenas nas tuplas que satisfaçam a condição;
- **Group by:** permite criar grupos de tuplas em um comando `SELECT` que utiliza várias tabelas. O `Group by` permite que funções sejam utilizadas para retornar valores em grupos;
- **Order by:** define uma ordem para a consulta. `ASC` realiza uma ordem ascendente e `DESC`, uma ordem descendente. Se não for utilizada alguma destas opções, o comando `SELECT` assumirá `ASC`.

Funções predefinidas |||||

As funções predefinidas são utilizadas para realizar operações entre as tuplas das tabelas. Elas retornarão um valor como resultado

da operação. Como exemplo, podemos citar a soma dos valores de uma coluna da tabela; isto pode ser facilmente realizado pela função `sum()` da SQL. Basta utilizar a função `sum(<nome da coluna>)` em conjunto com a cláusula `Group by <nome da mesma coluna>` no mesmo comando `SELECT`. Para que estas funções possam ser utilizadas, deve-se aproveitar a cláusula `Group by` para a coluna envolvida na operação. Veja quais são as funções predefinidas da SQL.

- `SUM(<Coluna>)`: retorna a soma dos valores da coluna;
- `AVG(<Coluna>)`: retorna a média dos valores da coluna, ignorando os valores nulos;
- `MIN(<Coluna>)`: retorna o menor valor encontrado na coluna;
- `MAX(<Coluna>)`: retorna o maior valor encontrado na coluna;
- `COUNT(<Coluna>) <Valor>`: retorna quantas vezes o valor especificado é encontrado na coluna.

Você utilizará funções predefinidas em nossos exemplos mais adiante.

UPDATE ||

Este comando é utilizado para atualizar o valor de uma tupla em uma tabela. Não é necessário atualizar todas as colunas, apenas aquela que tiver o valor a ser atualizado. Veja sua sintaxe:

```
UPDATE <Tabela>
Set
    <Coluna> = <Novo valor>,
    <Coluna> = <Novo valor>...
where <Condição>
;
```

Observe que este comando utiliza uma cláusula `where`, que deve ser incluída no comando para definir uma condição para a alteração dos dados da tabela.

Caso a cláusula `where` não seja utilizada, o `UPDATE` executará em todas as tuplas da tabela; isto jamais deve ser feito, ao menos que seja necessário.

Neste capítulo, foram apresentados os comandos SQL. A partir do próximo capítulo, será visto, na prática, o uso destes comandos para a criação e manutenção de bancos de dados no MySQL.

Capítulo 12

Criando e ativando um banco de dados no MySQL

Podemos criar um bancos de dados no HeidiSQL de duas maneiras:

- clicando com o botão direito sobre o ícone da conexão e escolhendo **Create database...**:

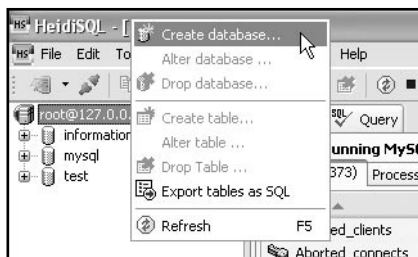


Figura 12.1: Criando um novo banco de dados no HeidiSQL.

- utilizando o comando `CREATE DATABASE` na Query Manager. Para executar o comando, basta digitá-lo e clicar na seta azul acima da área de comandos da Query Manager, ou utilizar a tecla F9 para executar o comando rapidamente. Crie um banco de dados de nome *mycontas*:

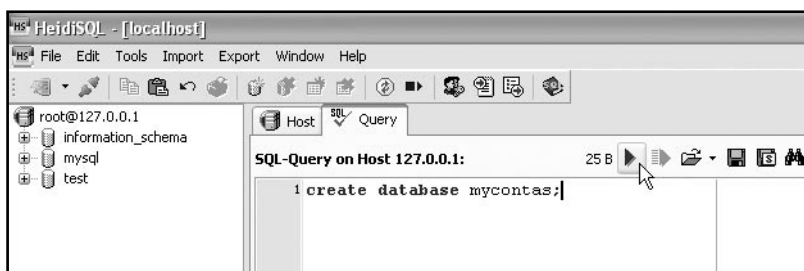


Figura 12.2: Criando um banco de dados utilizando a SQL na Query Manager.

O banco de dados *mycontas* será o que utilizaremos em nossos exemplos. Ao teclar F9, você perceberá que este banco de dados ainda não está na lista de bancos de dados da conexão; isto ocorre porque será necessário executar um *refresh*, ou seja, uma atualização na conexão com o banco de dados.

Isto pode ser realizado da seguinte forma:

1. Selecione a conexão root@127.1.1.1.
2. Execute o menu **Tools > Refresh**, ou utilize o botão **Refresh**, da barra de ferramentas, conforme mostra a **Figura 12.3**:

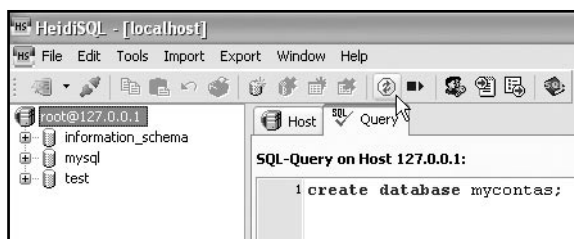


Figura 12.3: Executando o *refresh* no banco de dados com a barra de ferramentas.

Após atualizar, veja como o banco de dados aparecerá na conexão:

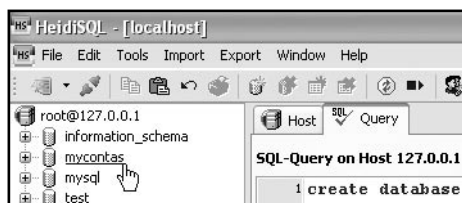


Figura 12.4: Banco de dados na lista após a execução do comando.

Capítulo 13

Excluindo um banco de dados

Na criação de um banco de dados, pode haver a necessidade de excluí-lo, por motivos de mudanças ou correções, visto que um banco de dados ativo há algum tempo jamais deve ser excluído, a não ser que suas informações sejam transferidas para outro banco.

Vamos criar outro banco de dados, de nome *olddb*, apenas para usar em nosso exemplo. Execute o comando a seguir, na Query Manager. Em seguida, atualize a conexão.

```
CREATE DATABASE olddb;
```

Caso seja necessário excluir um banco de dados, isto também poderá ser feito de duas formas:

- utilizando o menu *short cut* (o que aparece quando clicamos com o botão direito do mouse sobre o objeto). Clique com o botão direito do mouse sobre o banco de dados, escolhendo o comando **Drop Database** e clicando em **OK**, na mensagem de confirmação que aparecerá a seguir:

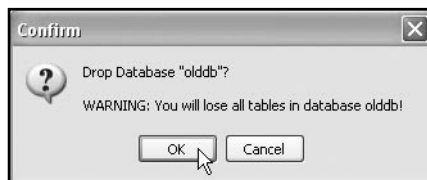


Figura 13.1: Confirmando a exclusão de um banco de dados.

- utilizando o comando `DROP DATABASE` na Query Manager. Neste caso, para excluir o banco de dados *olddb*, será usado o seguinte comando na Query Manager:

```
DROP DATABASE olddb;
```

Veja o resultado na **Figura 13.2:**

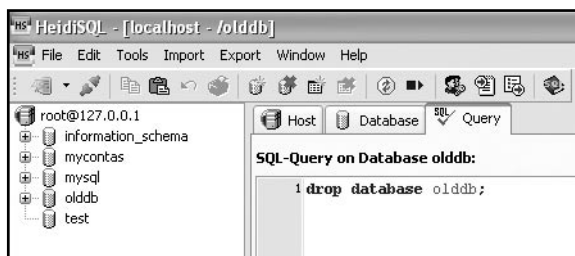


Figura 13.2: Excluindo um banco de dados utilizando a SQL na Query Manager.

O MySQL possui seus bancos de dados padrões, que não devem ser excluídos! Estes bancos de dados são os seguintes:

- information_schema;
- mysql;
- test.

Observação: como o HeidiSQL oferece uma interface simples e fácil de ser manuseada pelos usuários, pode ser perigoso fazer testes em uma máquina que tenha bancos de dados importantes e ativos. Por isso, muito cuidado com os comandos que você vai utilizar no HeidiSQL. Não modifique bancos de dados que estão em uso!

Capítulo 14

Criando tabelas

As tabelas de um banco de dados são estruturas de armazenamento. Elas são criadas para armazenar um conjunto de valores que serão utilizados posteriormente para gerar informações em um sistema. A forma mais fácil de criar tabelas no HeidiSQL é utilizando a Query Manager.

Observação: todos os comandos SQL dos exemplos deste livro serão executados na Query Manager.

No **Capítulo 4** foi definido um MER para um sistema de controle pessoal. Utilizaremos este MER para criar nossas tabelas em nosso banco de dados. Para isso, a tabela a seguir apresentará as principais referências entre o MER e o banco de dados no MySQL:

MER	MySQL
Entidade	Tabela
Atributo	Coluna
Tupla	Linha

Tabela 14.1.

Então, para cada entidade no MER será criada uma tabela no MySQL e, analogamente, para cada atributo de entidade do MER será criada uma coluna de tabela no MySQL. As tuplas representam as linhas da tabela.

Para criar uma tabela no MySQL, precisamos, inicialmente, selecionar o banco de dados que terá esta tabela. Para isso, basta clicar com o botão esquerdo do mouse sobre o banco de dados, no nosso caso, o *mycontas*. O banco de dados ativo estará com uma cor amarela, ficando, assim, em destaque com relação aos outros bancos de dados. Veja a **Figura 14.1**:

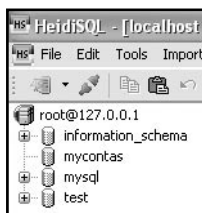


Figura 14.1: Selecionando uma tabela.

Observação: a partir de agora, utilizaremos comandos na Query Manager sempre com o banco de dados *mycontas* ativo (selecionado).

Já com o banco de dados selecionado, na Query Manager execute o seguinte comando:

```
CREATE TABLE CaixaMovimento
(
    LancamentoCaixa integer primary key
        not null auto_increment,
    CodigoHistorico integer(3)
        not null default 0,
    Documento varchar(12)
        not null default "",
    Data date not null,
    Valor real(7,2)
        not null default 0.00
)
Type = InnoDB
;
```

O comando anterior criará a tabela `CaixaMovimento` no banco de dados *mycontas*. Para criar as outras tabelas deste banco de dados, conforme o MER do **Capítulo 4**, digite e execute os comandos a seguir, na Query Manager:

```
CREATE TABLE BancoMovimento
(
    LancamentoBanco integer primary key
        not null auto_increment,
    Conta real(12,1) not null,
    LancamentoCaixa integer not null,
    Data date not null,
    TipoLancamento char(1)
        not null default "",
    Valor real(7,2)
        not null default 0.00
)
Type = InnoDB
;
```

```

CREATE TABLE HistoricoPadrao
(
    CodigoHistorico integer(3) primary key
        not null auto_increment,
    Descricao varchar(80)
        not null default ""
)
Type = InnoDB
;

```

```

CREATE TABLE ContaBanco
(
    Conta real(12,1) primary key
        not null,
    Agencia real(5,1)
        not null default 0.0,
    NomeBanco varchar(40)
        not null default ""
)
Type = InnoDB
;

```

```

CREATE TABLE ContasPagarReceber
(
    LancamentoContPG integer primary key
        not null auto_increment,
    CodigoHistorico integer(3)
        not null default 0,
    DataVencimento date not null,
    Documento varchar(12)
        not null default "",
    Valor real(7,2)
        not null default 0.00,
    Tipo char(2)
        not null default "",
    Situacao char(2)
        not null default ""
)
Type = InnoDB
;

```

Atualize a conexão e abra o link do banco de dados (clcando no sinal + ao lado do seu nome). Ao selecionar a tabela, você poderá clicar em **Table** para ver suas propriedades e para visualizar seus dados, neste momento ainda nenhum. Utilize a guia **Data**, ao lado da Query Manager. Veja as **Figuras 14.2 e 14.3**:

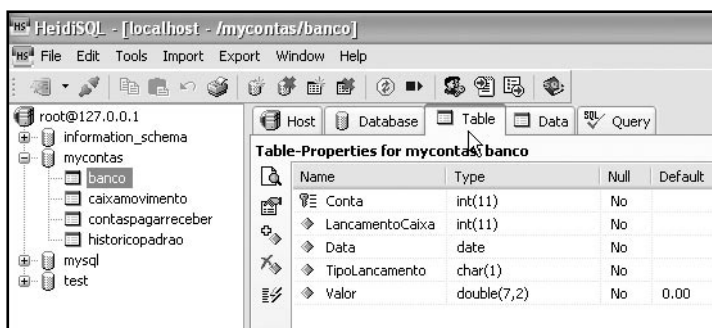


Figura 14.2: Visualizando as propriedades da tabela.

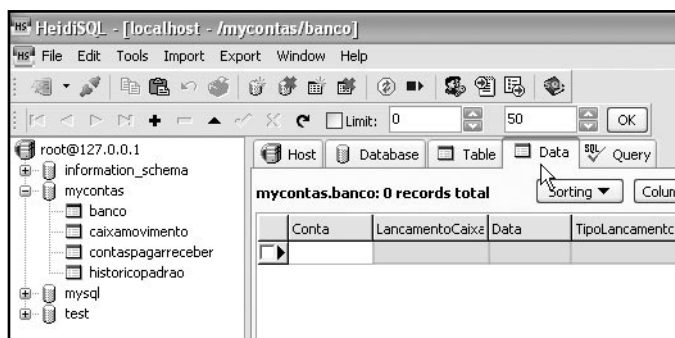


Figura 14.3: Visualizando dados da tabela.

Agora, você já está com o banco de dados *mycontas* e suas tabelas criadas no MySQL. Este banco de dados será utilizado em nossos exemplos mais adiante.

Capítulo 15

Excluindo tabelas

Tabelas também podem ser facilmente excluídas em um banco de dados do MySQL, por isto esta operação só deve ser realizada se você tiver absoluta certeza de que os dados da tabela não serão mais necessários.

Caso seja preciso excluir uma tabela do banco de dados, isto pode ser feito via SQL ou pelo menu *short cut*, ações muito parecidas com as empregadas para a exclusão de bancos de dados, como vimos no **Capítulo 13**.

Apenas para testar os comandos de exclusão de tabelas no banco de dados, crie uma tabela qualquer, de nome *oldtable*. A tabela *oldtable* deve ser excluída por uma das ações a seguir:

- utilizando o menu *short cut*. Clique com o botão direito do mouse sobre a tabela *oldtable*, escolha o comando **Drop table** e marque **OK** na mensagem de confirmação que aparecerá em seguida:

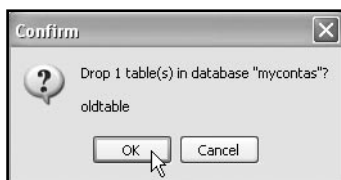


Figura 15.1: Confirmando a exclusão da tabela *oldtable*.

- utilizando o comando `DROP TABLE` na Query Manager. Neste caso, para excluir a tabela *oldtable* do banco de dados *mycontas* será usado o seguinte comando na Query Manager:

```
DROP TABLE oldtable;
```

Veja como isso ficaria na **Figura 15.2**:

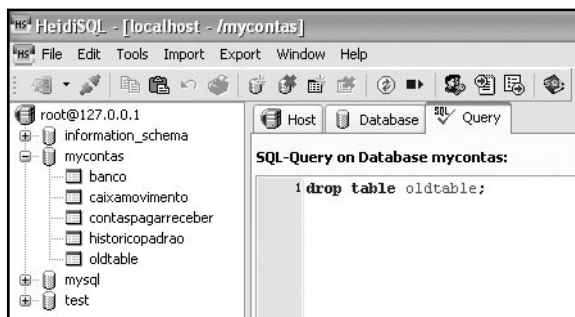


Figura 15.2: Excluindo a tabela *oldtable* com a SQL na Query Manager.

Quando for utilizado um comando SQL, execute-o na Query Manager e, logo em seguida, atualize a conexão para ver o resultado do comando. Se tudo ocorrer bem, o banco de dados *mycontas* não terá mais a tabela *oldtable*.

Capítulo 16

Bloqueando e desbloqueando tabelas

As tabelas de um banco de dados às vezes necessitam de manutenção durante a execução do sistema. Quando é feita a manutenção, executar operações nas tabelas não é uma ação adequada, pois poderá causar alguma falha.

Podemos utilizar os comandos `LOCK TABLES` e `UNLOCK TABLES` para efetuar a operação de bloqueio e desbloqueio de tabelas. Porém, o que é um bloqueio de tabelas em um banco de dados? Quando é feito um bloqueio em uma ou mais tabelas no banco de dados, elas ficarão protegidas contra gravação e exclusão de dados. Esta proteção irá durar até a execução do comando `UNLOCK TABLES`. Veja a sintaxe dos comandos citados:

```
LOCK TABLES
  <tabela> Read,
  <tabela> Read...
;
```

```
UNLOCK TABLES;
```

A cláusula `Read` do comando `LOCK TABLES` permite que as tabelas estejam liberadas para consultas por meio do comando `SELECT`, enquanto ficam bloqueadas para gravação e exclusão. Selecione o banco de dados *mycontas* e digite e execute o comando a seguir na Query Manager, para testar os conceitos aqui apresentados:

```
LOCK TABLES
  BancoMovimento Read,
  ContaBanco Read,
  CaixaMovimento Read,
  ContasPagarReceber Read,
  HistoricoPadrao Read
;
```

O comando anterior bloqueará todas as tabelas do banco de dados *mycontas*. Neste momento, estas tabelas não poderão ser utili-

zadas para gravação ou exclusão de dados. Para desbloquear novamente as tabelas, execute o comando `UNLOCK TABLES`:

```
UNLOCK TABLES;
```

Capítulo 17

Modificando uma tabela existente

Quando um sistema é implementado, ele é, normalmente, direcionado a atender um determinado segmento ou atividade. Como novas necessidades podem surgir a cada momento, uma adaptação pode ser necessária ao longo dos dias em que este sistema estiver sendo utilizado pelos seus usuários. Durante mudanças e adaptações em um sistema, é comum mudanças na estrutura de tabelas do banco de dados.

A modificação de uma tabela no MySQL pode ser realizada com a utilização do comando `ALTER TABLE`. Vamos, então, testar estes comandos. Será utilizada a tabela *historicopadrao* do banco de dados *mycontas*.

Adicionando colunas

Selecione o banco de dados *mycontas* e execute o comando a seguir na Query Manager, para adicionar novas colunas na tabela *historicopadrao*:

```
ALTER TABLE historicopadrao
  Add column DebitoCredito varchar(1),
  Add column Valor Real
;
```

Execute a opção **Refresh**, selecione a tabela *historicopadrao* e clique na guia **Table**, para ver as colunas *debitocredito* e *valor*, adicionadas em *historicopadrao*.

Modificando colunas

Agora, vamos modificar a coluna *debitocredito* da tabela do exemplo anterior. Execute o comando a seguir:

```
ALTER TABLE historicopadrao
  Modify column DebitoCredito char(3)
;
```

É só executar o **Refresh** e conferir o resultado na guia **Table**.

Removendo colunas

Serão removidas as colunas *debitocredito* e *valor* da tabela *historicopadrao*, visto que elas foram incluídas apenas para exemplificar o comando `ALTER TABLE`. Execute o comando a seguir, para que estas colunas sejam removidas.

```
ALTER TABLE historicopadrao
  Drop column debitocredito,
  Drop column valor
;
```

Capítulo 18

Inserindo novos dados nas tabelas

A inserção de novos dados em uma tabela de banco de dados é uma das operações mais realizadas por um sistema, perdendo apenas para a de leitura de dados. Felizmente, a SQL oferece um comando, o `INSERT`, com uma sintaxe muito objetiva para a realização destas operações.

O comando `INSERT` será utilizado sempre que for necessário incluir uma nova tupla (linha) em uma tabela no banco de dados. Este comando tem uma particularidade que merece atenção, a inclusão de valores em colunas previamente declaradas como `auto_increment`: estas colunas não devem ter seus valores incluídos manualmente, nem terem seus nomes referenciados em um comando `INSERT`, pois seus valores serão incluídos automaticamente pelo SGBD.

O comando `INSERT` tem duas cláusulas: `into` e `Values`. A cláusula `into` define a tabela com suas respectivas colunas, que receberão os valores definidos pela cláusula `Values`. Vamos então revisar a sintaxe do `INSERT`:

```
INSERT
  into <nome da tabela>
    (
      <Coluna1>,
      <Coluna2>...
    )
  Values
    (
      <Valor da coluna 1>,
      <Valor da coluna 2>...
    )
;
```

Os comandos a seguir incluirão valores na tabela *historicopadrao* do nosso banco de dados *mycontas*. Execute estes comandos na Query Manager e veja os resultados de cada um nas respectivas tabelas.

```
INSERT
  into historicopadrao
    (
```

```

        descricao
    )
    Values
    (
        "Despesa com alimentação"
    )
;

Insert
into historicopadrao
    (
        descricao
    )
    Values
    (
        "Despesa com combustível"
    )
;

Insert
into historicopadrao
    (
        descricao
    )
    Values
    (
        "Despesa com material escolar"
    )
;

Insert
into historicopadrao
    (
        descricao
    )
    Values
    (
        "Serviço prestado"
    )
;

```

Após executar estes comandos e efetuar o **Refresh** na conexão, você deve visualizar as tuplas que foram inseridas na tabela:



The screenshot shows the HeidiSQL application window. The left sidebar displays a tree view of the database structure, with 'mycontas.historicopadiao' selected. The main window shows the table's data in a tabular format. The table has two columns: 'Codigo' and 'Descricao'. There are four records listed, with the first record (Codigo 1) selected. The status bar at the top indicates 'mycontas.historicopadiao: 4 records total'.

Codigo	Descricao
1	Despesa com alimentação
2	Despesa com combustível
3	Despesa com material escolar
4	Serviço prestado

Figura 18.1: Dados incluídos.

Capítulo 19

Operadores

Existem comandos na linguagem SQL que utilizam uma cláusula chamada `where`. Esta cláusula possibilita a utilização de expressões utilizadas como filtros para a ação de alguns comandos SQL.

Para a criação de filtros nos comandos SQL, será necessária a utilização de operadores relacionados na tabela a seguir:

Operadores	Significado
=	Igual.
>	Maior que.
<	Menor que.
>=	Maior ou igual a.
<=	Menor ou igual a.
And	E (Utilizado para juntar expressões).
Or	Ou (Utilizado para comparar expressões).
Not	Não.
In	Em (Indica local).
Null	Nulo.
Like	Valida expressões com dados semelhantes.

Tabela 19.1.

Não se preocupe com o uso prático destes operadores agora, apenas com o conceito de sua utilização. A prática dos operadores nos comandos SQL será abordada nos próximos capítulos. Vamos, então, a uma explicação mais detalhada de cada um destes operadores:

- `=`: faz uma comparação de igualdade. A expressão `valor=100` avalia se a coluna *valor* de uma tabela é igual a 100;
- `>`: avalia se um termo possui valor maior que outro. A expressão `valor>100` avalia se a coluna *valor* tem resultado maior do que 100;
- `<`: avalia se um termo possui valor menor que outro. A expressão `valor<50` avalia se a coluna *valor* é menor do que 50;
- `>=`: avalia se um termo possui valor maior ou igual a outro. A expressão `valor>=10` avalia se a coluna *valor* é maior ou igual a 10;
- `<=`: avalia se um termo possui valor menor ou igual ao de outro. A expressão `valor<=20` avalia se a coluna *valor* é menor ou igual a 20;
- `And`: permite juntar expressões, por exemplo – `valor > 100 AND valor < 200`. Com uma cláusula como esta, serão retorna-

das apenas tuplas que possuem valores maiores que 100 e menores que 200;

- **Or:** significa “ou”. Permite que duas ou mais condições sejam testadas, esperando resultado verdadeiro de qualquer uma delas. Veja o exemplo a seguir: `valor = 10 OR valor = 20`. Com uma cláusula como esta, serão retornadas apenas tuplas com valores iguais a 10 ou iguais a 20;

- **Not:** operador de negação. Ele inverte o valor de uma condição, ou seja, se a expressão `valor=10` retornar verdadeiro, a expressão `NOT valor=10` será falso;

- **In:** permite indicar um local, ou seja, os resultados de um comando `SELECT`;

- **NULL:** é um valor do tipo nulo. `NULL` não é exatamente um operador, mas, sim, um tipo de dado que representa valores não definidos;

- **Like:** permite a comparação com valores aproximados. A expressão `descricao like “Despesa%”` define uma comparação na qual os valores para a coluna *descricao* devem iniciar com “Despesa”. O sinal `%` representa que os valores em sua posição não importam para a condição. Este comando também será visto na prática no **Capítulo 22**.

Capítulo 20

Excluindo tuplas de tabelas

Assim como a inclusão de novas tuplas em uma tabela é uma operação muito comum em um SGBD, a exclusão de alguma tupla existente também será necessária, pois faz parte do conjunto de operações na manutenção de um banco de dados.

O comando utilizado para excluir tuplas em uma tabela no MySQL é o `DELETE`. Este comando possui a cláusula `from`, que define a tabela na qual a operação vai ser realizada, e `where`, que define a condição para a exclusão das tuplas.

Importante: o comando `DELETE` jamais deve ser utilizado sem a cláusula `where`, pois, neste caso, irá excluir todas as tuplas da tabela!

Os comandos a seguir incluirão novas tuplas na tabela *historico-padrao*, que serão excluídas a seguir pelo comando `DELETE`. Execute estes comandos na Query Manager.

```
INSERT
    into historicopadrao
    (
        descricao
    )
    Values
    (
        "Minha conta"
    )
;
```

```
INSERT
    into historicopadrao
    (
        descricao
    )
    Values
    (
        "Sua conta"
```

```
)
;
```

Após incluir estas tuplas, veja na **Figura 20.1** como ficará a tabela *historicopadrao*:

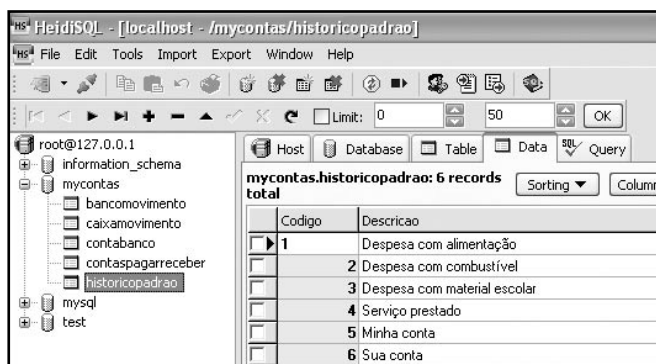


Figura 20.1: Novos dados em *historicopadrao*.

Agora, veja os comandos que serão utilizados para excluir as novas tuplas. Cada um destes testará o valor de uma coluna da tabela *historicopadrao*, para excluir ou não a tupla conforme a condição estabelecida pela cláusula *where*. Execute um a um os comandos a seguir, observando o resultado na guia **Data**, após selecionar a tabela *historicopadrao*:

```
DELETE
  from historicopadrao
  where CodigoHistorico=5
;

DELETE
  from historicopadrao
  where descricao="Sua conta"
;
```

Observe que a cláusula *where* define uma condição para exclusão das tuplas. Esta condição normalmente é uma comparação de valores estáticos, com valores armazenados nas colunas da tabela do banco de dados e, neste caso, serão deletados somente as tuplas que atendem esta condição. Para construir as condições da cláusula *where*, utilize os operadores apresentados no capítulo anterior.

Capítulo 21

Visualizando dados existentes

A visualização de dados no HeidiSQL pode ser feita de duas formas:

- por meio da guia **Data**, conforme a **Figura 21.1**, no fim desse capítulo;
- utilizando o comando `SELECT` da SQL.

A vantagem de utilizar o comando `SELECT` está na facilidade de visualizar todas as colunas de uma tabela, além das colunas de várias tabelas relacionadas, como veremos no **Capítulo 22**. Neste capítulo, faremos consultas simples nas tabelas do banco de dados, mas, para isso, será necessário incluir algumas tuplas na tabela *contaspagar-receber*, pois trabalharemos com ela agora. Execute os comandos a seguir na Query Manager:

```
INSERT
  into contaspagarreceber
  (
    CodigoHistorico,
    DataVencimento,
    Documento,
    Valor,
    Tipo
  )
Values
  (
    1,
    "2008.03.25",
    "NF. 142526",
    15.50,
    "CP"
  )
;
```

```
INSERT
  into contaspagarreceber
  (
    CodigoHistorico,
    DataVencimento,
```

```

        Documento,
        Valor,
        Tipo
    )
Values
    (
        1,
        "2008.03.26",
        "NF. 582451",
        52.70,
        "CP"
    )
;

INSERT
    into contaspagarreceber
    (
        CodigoHistorico,
        DataVencimento,
        Documento,
        Valor,
        Tipo
    )
Values
    (
        2,
        "2008.03.27",
        "NF. 361215",
        125.80,
        "CP"
    )
;

INSERT
    into contaspagarreceber
    (
        CodigoHistorico,
        DataVencimento,
        Documento,
        Valor,
        Tipo

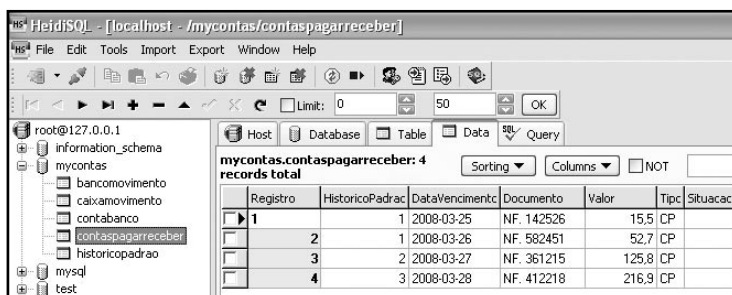
```

```

    )
Values
(
    3,
    "2008.03.28",
    "NF. 412218",
    216.90,
    "CP"
)
;

```

Veja na **Figura 21.1** como ficará a tabela *contaspagarreceber*, com estes dados incluídos:



The screenshot shows the HeidiSQL interface with the 'contaspagarreceber' table selected. The table has 4 records. The columns are: Registro, HistoricoPadrao, DataVencimento, Documento, Valor, Tipo, and Situacao.

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
1	1	2008-03-25	NF. 142526	15,5	CP	
2	1	2008-03-26	NF. 582451	52,7	CP	
3	2	2008-03-27	NF. 361215	125,8	CP	
4	3	2008-03-28	NF. 412218	216,9	CP	

Figura 21.1: Dados incluídos na tabela.

A coluna *Tipo* tem a função de definir se a conta é a pagar (CP) ou a receber (CR). A coluna *Situacao* destina-se ao registro da sigla PG, quando a conta é paga. Isto pode ser realizado por um sistema de gerenciamento de contas a pagar e a receber.

Para a visualização dos dados, vamos utilizar o comando `SELECT`. Para visualizar os dados de todas as colunas da tabela, utilize o comando a seguir:

```
SELECT * from contaspagarreceber;
```

O caractere `*` é utilizado para indicar que serão listadas todas as colunas da tabela. Os dados serão visualizados logo abaixo da Query Manager. Para visualizar apenas algumas colunas da tabela, adicione o nome das colunas no lugar do `*`. Veja como:

```
SELECT DataVencimento, Valor from contaspagarreceber;
```

Capítulo 22

Criando condições para visualização

No capítulo anterior, utilizamos o comando `SELECT` para visualizar os dados da tabela `contaspagarreceber`. Quando é utilizado o comando `SELECT` sem a cláusula `where`, serão relacionadas todas as tuplas existentes na tabela. No nosso caso, em que existem apenas quatro tuplas na tabela `contaspagarreceber`, o uso do comando `SELECT` sem uma condição para a visualização dos dados pode ser útil, mas para uma tabela com milhares de tuplas, pode ser impossível encontrar o dado necessário.

Para facilitar a pesquisa nas tabelas do banco de dados, o comando `SELECT` também utiliza a cláusula `where`, a mesma que utilizamos no comando `DELETE`. Quando utilizamos `where`, estamos definindo um filtro para o comando no qual somente as tuplas que atendam a condição serão retornadas na pesquisa. Execute os comandos a seguir e veja o resultado retornado por cada um deles:

- para visualizar as tuplas com valores menores que 100:

```
SELECT * from contaspagarreceber where valor < 100;
```

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
▶ 1	1	2008-03-25	NF. 142526	15,5	CP	
2	1	2008-03-26	NF. 582451	52,7	CP	

Figura 22.1: Tuplas com valor menor que 100.

- para visualizar as tuplas com valores maior ou igual a 200:

```
SELECT * from contaspagarreceber where valor >= 200;
```

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
▶ 4	3	2008-03-28	NF. 412218	216,9	CP	

Figura 22.2: Tuplas com valores maiores ou iguais a 200.

- para visualizar as tuplas com datas depois do dia 25 de cada mês, utilize a função `DAY()` da SQL, que retorna o dia de uma data. Isto pode ser feito conforme o comando a seguir:

```
SELECT * from contaspagarreceber where  
DAY(dataavencimento) > 25;
```

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
2	1	2008-03-26	NF. 582451	52,7	CP	
3	2	2008-03-27	NF. 361215	125,8	CP	
4	3	2008-03-28	NF. 412218	216,9	CP	

Figura 22.3: Tuplas com datas depois do dia 25 de cada mês.

- para visualizar as tuplas com valores entre 100 e 200:

```
SELECT * from contaspagarreceber where valor >= 100
AND valor <= 200;
```

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
3	2	2008-03-27	NF. 361215	125,8	CP	

Figura 22.4: Tuplas com valores entre 100 e 200.

- para visualizar as tuplas com valores menores que 100 e as tuplas de valores maiores que 200:

```
SELECT * from contaspagarreceber where valor < 100
OR valor > 200;
```

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
1	1	2008-03-25	NF. 142526	15,5	CP	
2	1	2008-03-26	NF. 582451	52,7	CP	
4	3	2008-03-28	NF. 412218	216,9	CP	

Figura 22.5: Tuplas com valores menores que 100 e maiores que 200.

- para visualizar as tuplas com valores não maiores que 100:

```
SELECT * from contaspagarreceber where NOT valor >
100;
```

Registro	HistoricoPadrao	DataVencimento	Documento	Valor	Tipo	Situacao
1	1	2008-03-25	NF. 142526	15,5	CP	
2	1	2008-03-26	NF. 582451	52,7	CP	

Figura 22.6: Tuplas que não têm valores maiores que 100.

Saiba mais: neste exemplo, o operador NOT aparece antes da expressão de comparação, invertendo seu resultado, ou seja, quando as tuplas forem maiores que 100, a expressão retornará verdadeiro, mas o operador NOT a transformará em falso, não permitindo seu retorno. Quando a expressão for negativa, ou seja, quando o valor for menor ou igual a 100, o operador NOT a transformará em verdadeiro, permitindo, assim, o retorno das tuplas na consulta.

- para exemplificar o operador `like`, vamos fazer uma consulta à tabela *historicopadrao*. O `like` é utilizado para definição de comparações aproximadas, vamos, então, listar as tuplas que possuem uma descrição começando com "Despesa":

```
SELECT * from historicopadrao where descricao like
"Despesa%";
```

Codigo	Descricao
1	Despesa com alimentação
2	Despesa com combustível
3	Despesa com material escolar

Figura 22.7: Histórico padrão com a descrição iniciada por "Despesa".

No exemplo anterior, serão mostradas as tuplas com descrição começando com a palavra "Despesa". Para encontrar tuplas que possuem um termo em comum, em qualquer posição, em uma determinada coluna, utilize o operador `like` conforme o exemplo a seguir, onde serão procuradas as tuplas que possuem o termo "ta", na coluna *descricao*:

```
Select * from historicopadrao where descricao like
"%ta%";
```

Codigo	Descricao
1	Despesa com alimentação
4	Serviço prestado

Figura 22.8: Histórico padrão que possui o termo "ta" em sua descrição.

Para maiores informações sobre as funções predefinidas da linguagem SQL, acesse o menu **Help > SQL Help**, ou pressione a tecla F1 no HeidiSQL. No item **Functions** do quadro **Topics**, você encontrará as funções desta linguagem, que poderão ser utilizadas nos comandos SQL. Neste livro, você verá as funções mais comuns e aprenderá como utilizar funções predefinidas, em conjunto com os comandos SQL.

Capítulo 23

Gerando consultas compostas

A construção de uma consulta composta existirá sempre que for necessário realizar um comando `SELECT` estruturado nos resultados de uma *query*; isto pode ser definido com a utilização de *subqueries*. O termo *query*, na SQL, significa “consulta”, ou seja, consulta a tuplas de tabelas em um banco de dados, por isso podemos dizer que o resultado de um comando `SELECT` é uma *query*.

Para realizar comandos com a utilização de *subqueries*, vamos incluir algumas tuplas em tabelas do banco de dados *mycontas*. Execute os comandos a seguir na Query Manager:

- incluindo dados na tabela *contabanco*:

```
INSERT
  into contabanco
    (
      Conta, Agencia, NomeBanco
    )
  Values
    (
      123456.7, 123.4, "Bradesco"
    )
;
```

```
INSERT
  into contabanco
    (
      Conta, Agencia, NomeBanco
    )
  Values
    (
      654321.8, 321.2, "Itaú"
    )
;
```

- incluindo dados na tabela *caixamovimento*:

```
INSERT
  into caixamovimento
```

```

        (
            CodigoHistorico, Documento, Data, Valor
        )
    Values
        (
            2, "DC145588", "2008-03-25", 145.70
        )
;

INSERT
    into caixamovimento
        (
            CodigoHistorico, Documento, Data, Valor
        )
    Values
        (
            3, "DC145589", "2008-03-26", 558.40
        )
;

```

- incluindo dados na tabela *bancomovimento*. A coluna *Tipo-Lancamento* desta tabela receberá os valores "D", para débito, ou "C", para crédito:

```

INSERT
    into bancomovimento
        (
            Conta, LancamentoCaixa, Data,
            TipoLancamento, Valor
        )
    Values
        (
            123456.7, 1, "2008-03-25", "D", 145.70
        )
;

INSERT
    into bancomovimento
        (
            Conta, LancamentoCaixa, Data,
            TipoLancamento, Valor

```

```

    )
  Values
  (
    123456.7, 2, "2008-03-26", "D", 558.40
  )
;

INSERT
  into bancomovimento
  (
    Conta, LancamentoCaixa, Data,
    TipoLancamento, Valor
  )
  Values
  (
    654321.8, 3, "2008-03-27", "D", 150.00
  )
;

```

Observação: note que foi incluída uma tupla na tabela *bancomovimento* que não corresponde a uma tupla da tabela *caixamovimento*. Lançamentos deste tipo devem ser evitados pelo sistema que irá alimentar o SGBD. Atualmente, restrições deste tipo são criadas em uma camada de Negócio, em um modelo em três camadas.

Para montar uma consulta composta, serão utilizados dois comandos `SELECT` básicos e o operador `IN`, empregado para indicar a *subquery* que contém os dados a serem pesquisados. Veja a seguir os comandos e a cláusula `where` que utilizaremos:

- `SELECT LancamentoCaixa from bancomovimento:` executa uma consulta de dados na tabela *bancomovimento* – esta consulta dependerá dos dados da *subquery* criada ainda neste comando;
- `SELECT Lancamento from caixamovimento:` *subquery* do comando;
- `where LancamentoCaixa IN:` condição que permitirá a exibição das tuplas apenas se os valores da coluna *LancamentoCaixa* estiverem na *subquery*;
- `where LancamentoCaixa NOT IN:` condição que permitirá a exibição das tuplas apenas se os valores da coluna *LancamentoCaixa* não estiverem na *subquery*.

A coluna que determinará se uma tupla da tabela *bancomovimento* corresponde a uma tupla da tabela *caixamovimento* é a coluna *LancamentoCaixa*. Esta coluna faz parte das duas tabelas e realiza um relacionamento entre elas. Este relacionamento pode ser visualizado no MER do **Capítulo 4**.

Vamos, então, aos comandos da nossa consulta composta. Primeiro, exibiremos as tuplas da tabela *bancomovimento* com lançamento na tabela *caixamovimento*. Depois, faremos o inverso, mediante a utilização do operador `NOT`, vamos mostrar apenas as tuplas da tabela *bancomovimento* que não têm um lançamento correspondente na tabela *caixamovimento*:

```
SELECT LancamentoCaixa
  from
    bancomovimento
 where
    LancamentoCaixa IN
    (SELECT LancamentoCaixa from caixamovimento);
```

O comando anterior listará duas tuplas, pois a condição `where` define uma consulta apenas para os registros da *subquery*. A ação deste comando, escrita em português, ficaria assim:

Selecione *LancamentoCaixa* em *bancomovimento* onde *LancamentoCaixa* esteja EM (SELECT *LancamentoCaixa* from *caixamovimento*);

Veja o resultado deste comando na **Figura 23.1**:

LancamentoCaixa
1
2

Figura 23.1: Tuplas comuns entre *bancomovimento* e *caixamovimento*.

Agora, execute o comando a seguir, o inverso do comando anterior. Neste comando, utilizaremos o operador `NOT` para listar as tuplas da tabela *bancomovimento* que não têm uma tupla correspondente na tabela *caixamovimento*:

```
SELECT LancamentoCaixa
  from
```

```
bancomovimento
where
LancamentoCaixa NOT IN
(SELECT LancamentoCaixa from caixamovimento);
```

O resultado deste comando está na **Figura 23.2**:



LancamentoCaixa
3

Figura 23.2: Tuplas da tabela *bancomovimento* que não têm correspondente na tabela *caixamovimento*.

Capítulo 24

Realizando consultas ordenadas

Para realizar consultas ordenadas em uma tabela do banco de dados, basta utilizar a cláusula `Order by`, do comando `SELECT`. Esta cláusula nos permite realizar consultas ordenadas de forma ascendente ou descendente, bastando, para isso, utilizar as opções `ASC` ou `DESC` da cláusula.

A seguir, vamos realizar alguns exemplos utilizando o comando `SELECT` com a cláusula `Order by`, para gerar consultas ordenadas. Execute estes comandos na Query Manager e veja o resultado de cada um deles:

- gerando uma consulta ordenada por descrição, na tabela *historicopadrao*:

```
SELECT CodigoHistorico, Descricao
  From historicopadrao
  Order by descricao;
;
```

Agora de forma descendente:

```
SELECT CodigoHistorico, Descricao
  From historicopadrao
  Order by descricao DESC;
;
```

- gerando consulta ordenada por valor, em *contaspagarreceber*:

```
SELECT LancamentoContPG, DataVencimento, Valor
  From contaspagarreceber
  Order by valor DESC;
;
```

Capítulo 25

Utilizando funções predefinidas

Como foi visto no **Capítulo 11**, a SQL possui várias funções predefinidas que podem ser utilizadas em consultas no banco de dados. Estas funções são rotinas que tendo o conhecimento de sua sintaxe já será usada, ou seja, não é necessário saber o que há no código destas funções; apenas o que elas fazem e como utilizá-las. Outra característica das funções está no seu retorno: elas sempre retornarão alguma coisa, pois é exatamente por causa dos seus retornos que as utilizamos.

As funções mais utilizadas na SQL são:

- `SUM(<Coluna>)`: retorna a soma dos valores da coluna;
- `AVG(<Coluna>)`: retorna a média dos valores da coluna, ignorando os valores nulos;
- `MIN(<Coluna>)`: retorna o menor valor encontrado na coluna;
- `MAX(<Coluna>)`: retorna o maior valor encontrado na coluna;
- `COUNT(<Coluna>)`: retorna quantas vezes valores não-nulos são encontrados na coluna.

Vamos ver um exemplo com cada uma delas:

- retornando uma soma dos valores da coluna *valor*, da tabela *contaspagarreceber*:

```
SELECT SUM(valor)
  from contaspagarreceber
;
```

- retornando a média da coluna *valor*, da tabela *contaspagarreceber*:

```
SELECT AVG(valor)
  from contaspagarreceber
;
```

- retornando o valor mínimo encontrado na coluna *valor*, da tabela *contaspagarreceber*:

```
SELECT MIN(valor)
  from contaspagarreceber
;
```

- retornando o valor máximo encontrado na coluna *valor*, da tabela *contaspagarreceber*:

```
SELECT MAX(valor)
  from contaspagarreceber
;
```

- retornando a quantidade de tuplas da tabela *contaspagarreceber*:

```
SELECT COUNT(LancamentoContPG)
  from contaspagarreceber
;
```

Execute estes comandos na Query Manager e veja o resultado.

Capítulo 26

Fazendo consulta em várias tabelas

Neste capítulo, veremos como realizar a consulta em mais de uma tabela do banco de dados. Isto pode ser realizado utilizando a cláusula `where` para filtrar os registros e montar uma tupla que vai além da estrutura de uma única tabela.

Como foi visto no MER do **Capítulo 4**, a tabela *historicopadiao* está relacionada com as tabelas *caixamovimento* e *contaspagarreceber* por meio da coluna *CodigoHistorico*. Quando um lançamento é realizado na tabela *contaspagarreceber*, será armazenado na coluna *CodigoHistorico* um código que corresponde a uma tupla da tabela *historicopadiao*, por isso, fica fácil montar uma consulta que liste as tuplas da tabela *contaspagarreceber* e, ao mesmo tempo, a descrição correspondente de cada histórico padrão desta tabela. Veja como isso fica no exemplo a seguir:

```
SELECT
    A.LancamentoContPG,
    A.DataVencimento,
    A.Documento,
    A.Valor,
    A.Tipo,
    B.Descricao
From
    contaspagarreceber A, historicopadiao B
Where
    A.CodigoHistorico = B.CodigoHistorico
;
```

Observe que no exemplo anterior utilizamos um apelido para cada tabela e identificamos as colunas das tabelas com a utilização dos seus respectivos apelidos; isto é necessário, pois utilizamos mais de uma tabela no comando. A coluna *CodigoHistorico*, neste caso, foi utilizada apenas para realizar o relacionamento, ou seja, listar as tuplas da tabela *historicopadiao*, que correspondem a cada tupla da tabela *contaspagarreceber*. Vamos à explicação de cada um destes itens:

- *A.LancamentoContPG*: utilizamos o apelido *A*, que representa a tabela *contaspagarreceber*, para identificar a sua coluna *LancamentoContPG*;

- B.Descricao: foi utilizado o apelido B, que representa a tabela *historicopadrazo*, para identificar a sua coluna de nome *Descrição*;
- From contaspagarreceber A, historicopadrazo B: definimos apelidos para as tabelas *contaspagarreceber* e *historicopadrazo*;
- Where A.CodigoHistorico = B.CodigoHistorico: condição para listar as tuplas das tabelas. Caso esta condição não seja utilizada neste comando, serão listadas todas as tuplas da tabela *historicopadrazo* para cada tupla da tabela *contaspagarreceber* e isso não faria nenhum sentido!

Veja o resultado deste comando na **Figura 26.1**:

	LancamentoContPG	DataVencimento	Documento	Valor	Tipo	Descricao
▶	1	2008-03-25	NF. 142526	15,5	CP	Despesa com alimentação
		2 2008-03-26	NF. 582451	52,7	CP	Despesa com alimentação
		3 2008-03-27	NF. 361215	125,8	CP	Despesa com combustível
		4 2008-03-28	NF. 412218	216,9	CP	Despesa com material escolar

Figura 26.1: Consultas nas tabelas *contaspagarreceber* e *historicopadrazo*.

Também podemos utilizar apelidos para o nome das colunas. Veja um exemplo com a utilização de apelidos nas colunas das tabelas:

```
SELECT
    A.LancamentoContPG as Reg,
    A.DataVencimento as Vencimento,
    A.Documento as Doc,
    A.Valor as Valor_da_conta,
    A.Tipo as Tipo_da_conta,
    B.Descricao as Histórico_Padrão
From
    contaspagarreceber A, historicopadrazo B
Where
    A.CodigoHistorico = B.CodigoHistorico
;
```

Como podemos ver neste exemplo, os títulos dos apelidos das colunas podem conter caracteres com acento e menos espaço entre nomes! Veja a figura:

Reg	Vencimento	Doc	Valor_da_conta	Tipo_da_conta	Histórico_Padrão
1	2008-03-25	NF. 142526	15,5	CP	Despesa com alimentação
2	2008-03-26	NF. 582451	52,7	CP	Despesa com alimentação
3	2008-03-27	NF. 361215	125,8	CP	Despesa com combustível
4	2008-03-28	NF. 412218	216,9	CP	Despesa com material escolar

Figura 26.2: Consultas nas tabelas *contaspagarreceber* e *historicopadrao* com a utilização de títulos nas colunas.

Agora, vamos a mais um exemplo de consulta com várias tabelas:

```

SELECT
    A.LancamentoBanco as Reg,
    A.Conta,
    B.NomeBanco as Banco,
    C.Data,
    C.Valor,
    D.Descricao as Histórico
From
    bancomovimento A, contabanco B,
    caixamovimento C, historicopadrao D
Where
    A.Conta = B.Conta AND
    A.LancamentoCaixa = C.LancamentoCaixa
AND
    C.CodigoHistorico = D.CodigoHistorico
;

```

Neste exemplo, utilizamos quatro tabelas e as colunas envolvidas nos relacionamentos para gerar a consulta, buscando dados em várias tabelas. O resultado da consulta está na figura seguinte:

Reg	Conta	Banco	Data	Valor	Histórico
1	123456,7	Bradesco	2008-03-25	145,7	Despesa com combustível
2	123456,7	Bradesco	2008-03-26	558,4	Despesa com material escolar

Figura 26.3: Consultas nas tabelas *bancomovimento*, *contabanco*, *caixamovimento* e *historicopadrao*.

Parte 4

MySQL avançado

Capítulo 27

Criando e gerenciando permissões de acesso TCP/IP

O MySQL oferece mecanismos que permitem definir restrições de acesso ao banco de dados a usuários e computadores da rede, mais conhecidos como hosts.

Para que um computador na rede possa ter acesso ao SGBD, será necessário adicionar um novo usuário e definir as permissões de acesso às tabelas do banco de dados. Para gerenciar estas permissões, o MySQL utiliza uma tabela de nome *user*, no banco de dados *mysql*. Veja, nas Figuras 27.1 e 27.2, este banco de dados e sua tabela *user*:

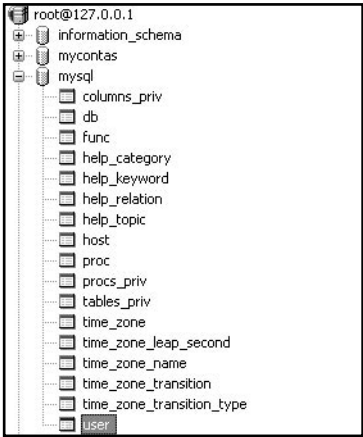


Figura 27.1: O banco de dados *mysql*.

	Host	User	Password	Select_priv	Insert_priv	Update_priv	Delete_priv	Create_priv	Drop_priv	Reload_priv	Shutdown_priv	Process
	localhost	root	"C1397073	✓	✓	✓	✓	✓	✓	✓	✓	✓

Figura 27.2: Tabela *user* do *mysql*.

Esta tabela possui três colunas iniciais, que definem o host, o usuário e a senha. As outras colunas definem privilégios do usuário. Quando estas colunas aparecem marcadas, significa que o privilégio a que estas colunas se referem está permitido para aquele usuário.

Antes de incluir usuários no MySQL, devemos observar algumas questões de segurança, vistas a seguir:

Dicas de segurança de informação

O MySQL exerce segurança por listas de controle de acesso (ACL). Para proteger seus dados, observe estas recomendações, embasadas nas dicas do próprio fabricante, principalmente para um banco de dados conectado à Internet:

- os usuários, exceto o *root*, não devem ter acesso ao banco de dados *mysql* ou ao comando `DROP`;
- não permita o acesso indiscriminado das tabelas dos bancos de dados. Permita acesso apenas às tabelas necessárias para aquele usuário;
- escolha senhas que não sejam palavras do dicionário;
- utilize *firewall*;
- não confie em dados digitados pelos usuários em aplicações do sistema. Um usuário mal-intencionado, por exemplo, pode entrar com a sequência `DROP DATABASE mysql`, em um formulário da Web. Por isso, teste todas as entradas de usuários antes de executá-las no banco de dados e não permita condições do tipo `Lancamento = Lançamento`, pois uma condição como esta no comando `DELETE` poderia excluir todas as tuplas da tabela, já que cada valor da coluna está sendo comparado consigo mesmo;
- não transmita dados sem criptografia pela Internet.

Alguns destes assuntos, como Firewall's, Criptografia SSL ou SSH, são de extrema importância à segurança dos sistemas e não estão relacionados exclusivamente ao MySQL, por isso, merecem um livro exclusivo, devido a sua extensão e complexidade. Porém, mesmo sem estudar a fundo estes protocolos, vamos ver algumas funções de criptografia que podem ser utilizadas no MySQL.

Criptografia no MySQL

Antes de adicionar novos hosts e permissões de acesso para o MySQL, vamos estudar as funções `AES _ ENCRYPT`, `AES _ DECRYPT` e `PASSWORD`, funções de criptografia e rotina de embaralhamento de caracteres, entre outras existentes, que podem ser utilizadas no MySQL:

AES _ ENCRYPT ||

Esta função permite transformar uma seqüência de caracteres em dado criptografado. A *Advanced Encryption Standard* tem vantagem sobre outros algoritmos de criptografia pelo fato de utilizar uma combinação que gera um valor difícil de ser descoberto. Esta combinação é uma chave (*String* definida pelo administrador) conhecida apenas pelo seu criador, ou seja, pelo usuário *root*. A sintaxe para criptografar uma seqüência com esta função está logo a seguir:

```
AES _ ENCRYPT(<String a ser criptografada>`,`<Chave>)
```

O termo <Chave> pode ser uma palavra qualquer, desde que seja de conhecimento exclusivo do usuário *root* e não seja palavra de dicionário.

AES _ DECRYPT ||

Esta função tem o papel de descriptografar uma seqüência criptografada pela função AES _ ENCRYPT. Sua sintaxe é a seguinte:

```
AES _ DECRYPT(<String a ser descriptografada>`,`<Chave>)
```

Para que a seqüência possa ser revertida, será necessário utilizar a mesma chave de criptografia usada na função AES _ ENCRYPT.

PASSWORD ||

Esta função não é bem uma de criptografia, mas é utilizada para embaralhar valores de senhas no MySQL. Sua sintaxe é a seguinte:

```
PASSWORD(<Senha a ser embaralhada>)
```

Capítulo 28

Criando um novo usuário

Para criar novos usuários no MySQL, podemos utilizar o menu **Tools > User-Manager** do HeidiSQL. Será aberta uma janela conforme a da **Figura 28.1**:



Figura 28.1: Janela User-Manager do HeidiSQL.

Para permitir o acesso ao SGBD por computador/usuário da rede, utilize o endereço IP daquela máquina para identificar o host. Para um exemplo de criação de novo usuário, vamos definir as configurações conforme a tabela a seguir:

Item	Valor
Username	Jannet.
From Host	127.0.0.1.
Password	123654.
With Grant Options	No (opção desmarcada).
All Privileges	No (opção desmarcada).
Sessão Privileges	Todas as opções desmarcadas.

Tabela 28.1.

Importante: observe que as senhas criadas nos exemplos não são seguras, pois foram criadas apenas para ilustrar os comandos. Utilize pelo menos 16 dígitos e caracteres especiais, números e letras para criar uma senha segura!

Após definir estas opções, clique no botão **Add User** para criar o novo usuário. A **Figura 28.2** mostra como devem ficar os dados após a definição da tabela anterior. A **Figura 28.3** mostra a janela de confirmação após a criação do usuário:

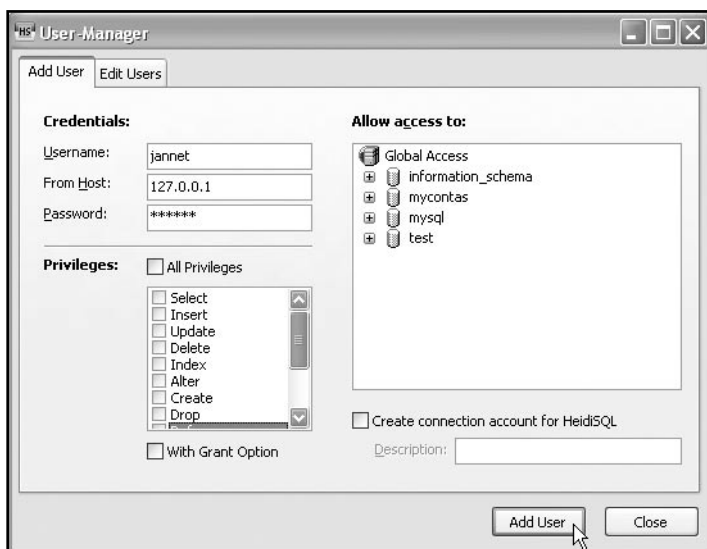


Figura 28.2: Criando um novo usuário no MySQL.

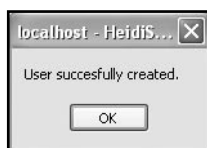


Figura 28.3: Janela de confirmação para a criação do novo usuário no SGBD.

Agora que o usuário já foi criado, devemos definir as permissões de acesso às tabelas dos bancos de dados. Isto não será feito na janela **User-Manager**, pois utilizaremos o comando `GRANT`, mais completo, e que nos habilita a criar permissões mais precisas para os usuários do banco de dados.

Clique no botão **Close**, na janela **User-Manager**, e pressione **Refresh** na conexão. Abra a tabela *user* do banco de dados *mysql*. Agora a tabela *user* possui o novo usuário de log-in *jannet*. Porém, observe que este usuário não tem ainda nenhum privilégio de acesso às tabelas do banco de dados:

	Host	User	Password	Select_priv	Insert_priv	Update_priv	Delete_priv	Create_priv
▶	localhost	root	*C13970736982BA019	☒	☒	☒	☒	☒
▶	127.0.0.1	jannet	*B9583F0F923655674	☐	☐	☐	☐	☐

Figura 28.4: Novo usuário incluído no MySQL.

Definindo privilégios de acesso

Para permitir que *jannet* tenha acesso somente às tabelas do banco de dados *mycontas*, com privilégios apenas para os comandos **SELECT**, **INSERT** e **UPDATE**, execute o comando a seguir na Query Manager:

```
GRANT SELECT, INSERT, UPDATE
ON mycontas.*
TO 'jannet'@'127.0.0.1';
```

O caractere @ foi utilizado para identificar o host em que o usuário está registrado. O termo `'jannet'@'127.0.0.1'` lê-se “jannet em 127.0.0.1”.

Após executar corretamente o comando, ao visualizar de novo os dados da tabela *user*, você verá que seus dados não mudaram, pois estes privilégios foram registrados em outra tabela, a tabela *db*, no banco de dados *mysql*.

Após executar o comando **GRANT**, será adicionado um novo privilégio à tabela *db* do banco de dados *mysql*. Abra, então, esta tabela e veja o privilégio adicionado com a utilização do comando **GRANT**, executado anteriormente. A **Figura 28.5** mostra os dados desta tabela:

	Host	Db	User	Select_priv	Insert_priv	Update_priv	Delete_priv	Create_priv	Drop_priv	Grant_priv	References_priv
▶	127.0.0.1	mycontas	jannet	☒	☒	☒	☐	☐	☐	☐	☐

Figura 28.5: Privilégio adicionado pelo comando **GRANT**.

Agora, ao logar com *jannet*, perceberá que somente o banco de dados *mycontas* estará disponível para uso dos comandos **SELECT**,

INSERT e UPDATE, na conexão deste novo usuário com o SGBD. A nova conexão está na figura seguinte:

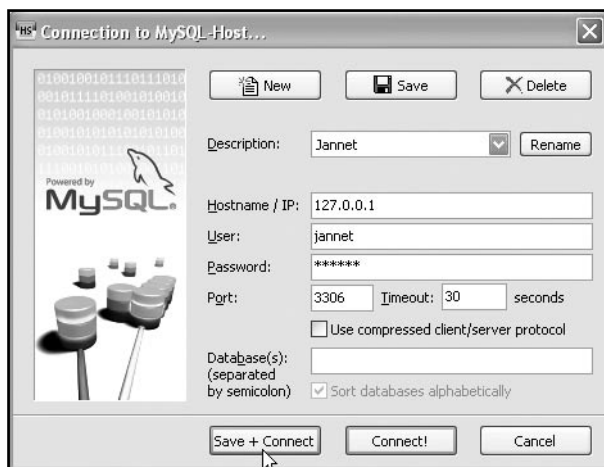


Figura 28.6: Conectando pela conta *jannet* ao MySQL.

Capítulo 29

Excluindo um usuário

A exclusão de usuários no MySQL pode ser realizada de forma muito simples. É importante lembrar que, para realizar esta e outras operações, será necessário estar logado como usuário *root*, pois apenas ele tem permissões para realizar comandos no banco de dados. Por exemplo, a conta *jannet*, criada no capítulo anterior, só tem permissões para realizar os comandos `SELECT`, `INSERT` e `UPDATE` no banco de dados *mycontas*.

Importante: a partir de agora, todos os exemplos deverão ser realizados em conexão por meio da conta *root* do MySQL.

Para exercitar, vamos criar uma nova conta, para que seja posteriormente excluída pelo comando `DROP USER`. Crie o usuário conforme a tabela a seguir:

Item	Valor
Username	novousuario.
From Host	127.0.0.1.
Password	123321.
With Grant Options	No (opção desmarcada).
All Privileges	No (opção desmarcada).
Seção Privileges	Todas as opções desmarcadas.

Tabela 29.1.

Após incluir o usuário, execute o botão **Refresh** na conexão e veja o novo usuário incluído na tabela *user*:

	Host	User	Select_priv	Password	Insert_priv
	localhost	root	☒	*C13970736982BA0190FE9AFD1CAF38566	☒
	127.0.0.1	novousuario	☐	*437F1809645E0A92DAB553503D2FE21D	☐
	127.0.0.1	jannet	☐	*B9583F0F92365567438EB552A2F7A815F	☐

Figura 29.1: Novo usuário incluído no MySQL.

Para excluir este novo usuário, execute o comando a seguir na Query Manager:

```
DROP USER 'novousuario'@'127.0.0.1';
```

Após executar o comando anterior e o **Refresh** na conexão com o banco de dados, você verá que a conta *novousuario* não estará mais na tabela *user* do banco de dados *mysql*:

	Host	User	Password	Select_priv	Insert_priv
☒	localhost	root	*C13970736982BA0190FE9AFD1CAF3856B971EAE	☒	☒
☐	127.0.0.1	jannet	*B9583F0F92365567438EB552A2F7A815FE0BCF3	☐	☐

Figura 29.2: Tabela *user* após exclusão de *novousuario*.

Capítulo 30

Modificando senhas de usuários

Para modificar uma senha de um determinado usuário em primeiro lugar devemos estar logados com o *root*. Após conectar-se com o MySQL pela conta *root*, podemos definir uma nova senha para um usuário do sistema de forma muito simples e objetiva: basta utilizar o comando `SET PASSWORD`.

Para testar este comando, vamos alterar a senha da conta *jannet*, que criamos anteriormente. Execute o comando a seguir para alterar a senha da conta *jannet*:

```
SET PASSWORD
  for 'jannet'@'127.0.0.1'
  = '654321'
```

Após realizar uma operação em uma conta ou privilégio dos usuários do sistema, utilize o menu **Tools > Flush > Privileges** do HeidiSQL, para realizar um *reload* (releitura) nos privilégios do SGBD. Este comando tem o resultado parecido com o de atualização, que já utilizamos vastamente em nossos exemplos.

Após alterar a senha da conta *jannet* e executar o **Flush Privileges** no MySQL, conecte novamente à conta *jannet*, agora com a nova senha. Para gravá-la na janela de conexão, após digitá-la, utilize o botão **Save + Connect**.

Capítulo 31

Criando *stored procedures*

Você já deve ter ouvido o termo *stored procedure*, isto se ainda não trabalhou com algumas delas! *Stored procedures* são rotinas gravadas dentro de um SGBD; são como um pedaço de código de programa que fica armazenado para ser posteriormente utilizado. Em outras palavras, *stored procedures*, em um SGBD, são rotinas criadas para serem posteriormente utilizadas, facilitando, assim, a vida dos administradores do sistema ou usuários.

Vamos, então, criar uma *stored procedure* que retornará a quantidade de tuplas de cada tabela do banco de dados *mycontas*. O comando utilizado para criar uma *stored procedure* está logo a seguir. Veja sua sintaxe:

```
CREATE PROCEDURE <banco de dados>.<nome da rotina>
  (Parâmetros de entrada e saída)
  BEGIN
    <Comandos Select Into>
  END;
```

Criando uma *stored procedure* passo a passo

Para criar uma *stored procedure*, vamos ter de realizar algumas operações, como, por exemplo, a mudança dos delimitadores de comando do MySQL antes de executar o comando de criação da rotina.

A cada passo da criação da rotina que executará comandos `SELECT` em várias tabelas, vamos armazenar os resultados em variáveis:

1. Antes de criar a rotina, é necessário mudar o delimitador de comandos do MySQL, pois no conteúdo da rotina existe o caractere `;`, que não pode ser confundido pelo sistema como uma finalização de comando. Execute o comando a seguir para mudar o delimitador:

```
Delimiter //
```

2. Este comando irá mudar o delimitador do MySQL para `//`. Isto deve ser feito somente para execução do comando `CREATE PRO-`

CEDURE. Mais adiante, voltaremos o delimitador para o caractere padrão (;).

3. Na Query Manager, execute o comando a seguir para criar a rotina que irá retornar a quantidade de tuplas de cada tabela do banco de dados *mycontas*:

```
CREATE PROCEDURE mycontas.QuantasTuplas
(
    OUT vContaBanco int,
    OUT vHistoricoPadrao int,
    OUT vCaixaMovimento int,
    OUT vBancoMovimento int,
    OUT vContPagarReceber int
)
BEGIN
    Select Count(Conta)
        into vContaBanco from contabanco;
    Select Count(CodigoHistorico)
        into vHistoricoPadrao from historicopadrao;
    Select Count(LancamentoCaixa)
        into vCaixaMovimento from caixamovimento;
    Select Count(LancamentoBanco)
        into vBancoMovimento from bancomovimento;
    Select Count(LancamentoContPG)
        into vContPagarReceber from contaspagarreceber;
END;
```

4. Este comando define algumas variáveis de saída, determinadas pelo parâmetro OUT. Estas variáveis terão seus valores incrementados pelos comandos SELECT entre a estrutura BEGIN .. END, quando a rotina for executada. Veja os detalhes deste comando:

- CREATE PROCEDURE mycontas.QuantasTuplas: cria uma *stored procedure* de nome *QuantasTuplas* no banco de dados *mycontas*;
- OUT vContaBanco int: define uma variável de nome vContaBanco que terá seu valor incrementado por um comando SELECT into;
- BEGIN .. END: estrutura da *stored procedure* que contém os comandos a serem executados;

- `SELECT Count(Conta) into vContaBanco from conta-banco;` executa um cálculo na coluna *Conta*, da tabela *contabanco*, do banco de dados *mycontas* e armazena o valor na variável *vContaBanco*.

5. Após executar este comando na Query Manager, visualize a tabela *ROUTINES* do banco de dados *information_schema*. Agora, a *stored procedure* *QuantasTuplas* deve estar registrada na tabela, conforme a **Figura 31.1**:

SPECIFIC_NAME	ROUTINE_CATALOG
QuantasTuplas	

Figura 31.1.

6. Podemos voltar o delimitador para o padrão: execute o comando a seguir na Query Manager:

```
Delimiter ;
```

7. Agora que a rotina já está criada e incluída no banco de dados, para executá-la e armazenar os valores em variáveis, basta utilizar o comando `CALL`. Execute o comando a seguir, para executar a *stored procedure* *QuantasTuplas* do banco de dados *mycontas*:

```
CALL mycontas.QuantasTuplas
(
    @TuplasContaBanco,
    @TuplasHistorico,
    @TuplasCaixa,
    @TuplasBancoMov,
    @TuplasContasPR
)
```

8. Após executar o comando `CALL`, a *procedure* *QuantasTuplas* será executada e as variáveis de retorno dela, definidas como `OUT`, serão vinculadas às variáveis definidas no comando `CALL`, por meio do operador `@`. Após executar este comando, você ainda não visualizará os dados, pois será necessário, agora, utilizar o comando `SELECT` para visualizar os dados retornados pela *stored procedure* *QuantasTuplas*.

9. Para visualizar os dados retornados pela *procedure* `QuantasTuplas`, execute o comando:

```
Select
    @TuplasContaBanco,
    @TuplasHistorico,
    @TuplasCaixa,
    @TuplasBancoMov,
    @TuplasContasPR
```

10. Ao executar este `SELECT`, você estará listando os valores das variáveis definidas pelo comando `CALL`, as quais já possuem os valores retornados pela *stored procedure*. O resultado deste comando `SELECT` está na **Figura 31.2**:

@TuplasContaBanco	@TuplasHistorico	@TuplasCaixa	@TuplasBancoMov	@TuplasContasPR
2	4	2	3	4

Figura 31.2: Valores retornados pela *stored procedure* `QuantasTuplas`.

Capítulo 32

Criando *triggers* (gatilhos)

Trigger é um tipo de *stored procedure* que possui mecanismos para ser executada de forma automática, e por isso é chamada de “gatilho”. A sua execução vai depender de um evento no banco de dados, ou seja, um comando SQL executado em uma determinada tabela.

A *trigger* tem estrutura parecida com a de uma *stored procedure* comum, contendo comandos SQL dentro da estrutura `BEGIN .. END`, mas ela possui algumas cláusulas que definirão em qual evento ela será disparada. Veja a sintaxe de uma *trigger*:

```
CREATE TRIGGER <Nome da Trigger> <Quando> <Evento>
  On <Banco de dados>.<Tabela>
  For <Abrangência>
  BEGIN
    <Comandos SQL>
  END;
```

Vamos aos detalhes deste comando:

- <Nome da Trigger>: define o nome da *trigger*;
- <Quando>: define quando a *trigger* será disparada. Esta opção pode ser `BEFORE` (antes do evento) ou `AFTER` (depois do evento);
- <Evento>: evento ocorrido na tabela. Pode ter os valores `INSERT`, `UPDATE` ou `DELETE`;
- `On <Banco de dados>.<Tabela>`: define o banco de dados e a tabela a que a *trigger* se refere;
- `For <Abrangência>`: deve ser definido como `FOR EACH ROW`. Isto significa que o gatilho disparará para cada linha incluída, excluída ou alterada na tabela.

Criando uma *trigger* passo a passo

Vamos a um exemplo de criação de *trigger*:

1. Antes de criar a rotina, é necessário mudar o delimitador de comandos do MySQL. Execute o comando a seguir para mudar o delimitador:

Delimiter //

2. Antes de criar a *trigger*, vamos criar uma tabela para armazenar os lançamentos indevidos em *bancomovimento*. Execute o comando a seguir na Query Manager para criar esta tabela:

```
Create table mycontas.LctoOrfao
(
    Reg int primary key not null auto_increment,
    DataLcto date,
    HoraLcto time,
    LancamentoCaixa integer
);
```

3. Atualize a conexão e veja esta nova tabela, no banco de dados *mycontas*.

4. Na Query Manager, execute o comando a seguir, para criar a *trigger*:

```
CREATE TRIGGER BancoMovLctoOrfao AFTER INSERT
ON mycontas.bancomovimento FOR EACH ROW
BEGIN
    IF new.LancamentoCaixa not in
    (
        Select LancamentoCaixa
        from caixamovimento
        where LancamentoCaixa = new.Lancamento-
Caixa
    )
    THEN
        Insert into mycontas.LctoOrfao
        (
            DataLcto,
            HoraLcto,
            LancamentoCaixa
        )
        values
        (
            CURDATE(),
```

```

CURTIME(),
new.LancamentoCaixa
);
END IF;
END;

```

5. Vamos estudar a *trigger*:

- **CREATE TRIGGER BancoMovLctoOrfao:** cria uma *trigger* de nome *BancoMovLctoOrfao*;
- **AFTER INSERT:** declara que a ação da rotina será executada depois do evento **INSERT** na tabela;
- **ON mycontas.bancomovimento:** define a tabela na qual a *trigger* ficará vinculada;
- **IF new.LancamentoCaixa not in:** compara se o novo valor da coluna *LancamentoCaixa*, da tabela *bancomovimento*, não corresponde a um valor na mesma coluna da tabela *CaixaMovimento*;
- **Select LancamentoCaixa from caixamovimento where LancamentoCaixa = new.LancamentoCaixa:** busca tuplas da tabela *caixamovimento*, que tem o valor da coluna *LancamentoCaixa*, correspondente ao valor incluído na tabela *bancomovimento*;
- **THEN:** faz parte da estrutura **IF**. O código que estiver depois de **THEN** será executado caso a condição **IF** seja verdadeira;
- **Insert into mycontas.LctoOrfao...:** inclui uma tupla na tabela *LctoOrfao*, caso a condição do comando **IF** anterior seja verdadeira.

6. Já podemos voltar o delimitador para o padrão: execute o comando a seguir na Query Manager:

```
Delimiter ;
```

7. Neste momento, a *trigger* já está criada e pode ser visualizada na tabela *TRIGGERS* do banco de dados *information_schema*, após a execução do **Refresh**.

Agora que a *trigger* já está criada, vamos testar sua ação, incluindo uma tupla na tabela *bancomovimento*.

Execute o comando a seguir na Query Manager:

```
Insert
into bancomovimento
(
    Conta, LancamentoCaixa, Data,
    TipoLancamento, Valor
)
values
(
    123456.7, 3, "2008-03-25", "D", 145.70
)
;
```

Veja que ao incluir um lançamento na tabela *bancomovimento* que não corresponda a um lançamento na tabela *caixamovimento*, a *trigger* disparou o seu comando `Insert`, registrando a operação na tabela *LctoOrfao*, do banco de dados *mycontas*. Visualize os dados da tabela *LctoOrfao* e veja o registro gerado automaticamente pela *trigger* `BancoMovLctoOrfao`.

Capítulo 33

Criando um script de banco de dados compatível com o MySQL

Os scripts de bancos de dados são pequenos arquivos de texto, normalmente com extensão SQL. A função destes arquivos é armazenar comandos SQL que poderão ser utilizados novamente em um SGBD.

Imagine que você esteja desenvolvendo um sistema e queira, futuramente, mudar de SGBD ou, se for um sistema para Internet, mudar de provedor de hospedagem. Caso tenha um script de banco de dados, poderá implementar a estrutura do mesmo banco de dados novamente, sem nenhum esforço: basta rodar o script, em vez de executar novamente todos os comandos de criação e definição de dados.

Para criar um script com os comandos de definição do banco de dados e suas tabelas, digite os comandos a seguir na Query Manager e salve-os em um diretório de sua preferência. Para salvar o arquivo, basta utilizar o botão **Save SQL to Textfile**, da barra de ferramentas da Query Manager:

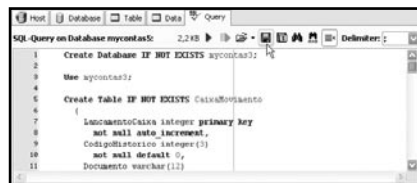


Figura 33.1: Salvando um script de banco de dados.

Os comandos do arquivo de script estão logo a seguir:

```
CREATE DATABASE IF NOT EXISTS mycontas;
```

```
Use mycontas;
```

```
Create Table IF NOT EXISTS CaixaMovimento  
(  
    LancamentoCaixa integer primary key  
        not null auto_increment,  
    Codigohistorico integer(3)  
        not null default 0,  
    Documento varchar(12)  
        not null default "",
```

```

        Data date not null,
        Valor real(7,2)
            not null default 0.00
    )
    Type = InnoDB
;

Create Table IF NOT EXISTS BancoMovimento
(
    LancamentoBanco integer primary key
        not null auto_increment,
    Conta real(12,1) not null,
    LancamentoCaixa integer not null,
    Data date not null,
    TipoLancamento char(1)
        not null default "",
    Valor real(7,2)
        not null default 0.00
)
Type = InnoDB
;

Create Table IF NOT EXISTS HistoricoPadrao
(
    CodigoHistorico integer(3) primary key
        not null auto_increment,
    Descricao varchar(80)
        not null default ""
)
Type = InnoDB
;

Create Table IF NOT EXISTS ContaBanco
(
    Conta real(12,1) primary key
        not null,
    Agencia real(5,1)
        not null default 0.0,
    NomeBanco varchar(40)
        not null default ""
)

```

```

    Type = InnoDB
;

Create Table IF NOT EXISTS ContasPagarReceber
(
    LancamentoContPG integer primary key
        not null auto_increment,
    CodigoHistorico integer(3)
        not null default 0,
    DataVencimento date not null,
    Documento varchar(12)
        not null default "",
    Valor real(7,2)
        not null default 0.00,
    Tipo char(2)
        not null default "",
    Situacao char(2)
        not null default ""
)
Type = InnoDB
;

Create table IF NOT EXISTS mycontas.LctoOrfao
(
    Reg int primary key not null auto_increment,
    DataLcto date,
    HoraLcto time,
    LancamentoCaixa integer
)
;

```

Observe que neste script utilizamos apenas os comandos que criam as estruturas do banco de dados e suas tabelas. Estes comandos possuem a cláusula `IF NOT EXISTS`, que permite que a estrutura do banco de dados e suas respectivas tabelas sejam criadas apenas se ainda não existirem no SGBD, evitando, assim, erros de execução do script. Recomendando que a criação das *stored procedures* seja feita em outro script, que deve ser executado somente uma vez!

Salve os scripts de banco de dados e guarde-os, pois pode necessitar deles um dia. Para rodar os comandos, após digitá-los na Query Manager e salvar o script, tecla F9.

Importante: este script irá apenas criar a estrutura do banco de dados. Caso seja necessário levar os dados armazenados nas tabelas para outro SGBD, eles devem ser migrados para o novo banco de dados, em rotina específica, em caso de mudança de SGBD! Lembre-se de que para rodar um script de banco de dados, ele deve estar compatível com o SGBD que irá executá-lo, ou seja, deve ser criado com os tipos de dados compatíveis ao SGBD. Veja no **Capítulo 35** como fazer backups dos dados no MySQL!

Caso queira abrir um script do MySQL, utilize o botão **Load SQL from Textfile**, na barra de ferramentas da Query Manager.

Capítulo 34

Manutenção do banco de dados

Às vezes, um SGBD pode apresentar algum problema nas tabelas do banco de dados. As tabelas seguras, InnoDB, por exemplo, têm maiores vantagens sobre as tabelas MyISAM com relação à integridade dos dados, mas, mesmo assim, é bom saber o que fazer caso uma delas venha a falhar.

O HeidiSQL oferece uma ferramenta para manutenção das tabelas do banco de dados. Selecione o banco de dados *mycontas* e abra o menu **Tools > Maintenance**, para selecionar a ferramenta de manutenção do HeidiSQL:

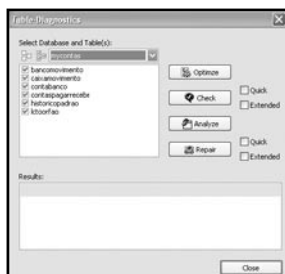


Figura 34.1: Janela de manutenção de tabelas do HeidiSQL.

Para realizar a manutenção das tabelas do banco de dados, selecione o banco de dados em **Select Database and Table(s)**. Marque a tabela na qual deseja realizar a manutenção e utilize um dos botões relacionados a seguir:

- **Optimize:** executa o comando `OPTIMIZE TABLE`, para desfragmentar a tabela. Uma tabela deve ser desfragmentada após muitas mudanças dos seus dados;
- **Check:** executa o comando `CHECK TABLE` para procurar erros na tabela;
- **Analyze:** executa o comando `ANALYZE TABLE` para analisar e redistribuir as chaves de tabela;
- **Repair:** executa o comando `REPAIR TABLE`, para reparar uma tabela possivelmente corrompida (quebrada).

Estes são os comandos utilizados para reparar tabelas no MySQL. Utilize a opção **Quick**, para execução rápida dos comandos, ou a opção **Extended**, para ler a tabela linha por linha.

Para executar estes comandos, você deve estar logado com o usuário *root*.

Capítulo 35

Fazendo backup dos dados

Uma das coisas mais importantes em um sistema é o backup dos dados. Backup é uma cópia de segurança que deve ser armazenada em um local seguro, para ser utilizada em uma eventual perda dos dados.

O MySQL oferece o aplicativo **mysqldump**, para efetuar o backup dos dados. Vamos, então, seguir passo a passo a criação de um backup válido para todos os bancos de dados do MySQL:

1. Efetue o log-off de todos os usuários do MySQL.
2. Acesse o menu **Iniciar** do Windows e abra o menu **Executar**.
3. Digite o comando `cmd` e clique em **OK**.
4. Acesse a pasta `C:\Arquivos de programas\MySQL\MySQL Server 5.0\bin`.
5. Execute o comando a seguir, todo na mesma linha do prompt:

```
mysqldump -u root -p --single-transaction --all-databases >
C:\Backup _ MySQL\Backup _ 20080403.sql
```

6. Quando o sistema pedir o *password*, digite a senha *senh%01*, conforme definido na instalação do MySQL.

Vamos estudar as opções utilizadas:

- `-u root`: loga com o usuário *root*;
- `-p`: pede senha do usuário;
- `--single-transaction`: define transação simples;
- `--all-databases`: backup para todos os bancos de dados;
- `C:\Backup _ MySQL\Backup _ 20080403.sql`: define local e nome do arquivo que o backup será armazenado.

Com tudo feito, o **mysqldump** efetuará o backup de todos os bancos de dados do MySQL. Observe que o aplicativo criou um script com todas as entradas, estruturas e dados dos bancos de dados do MySQL via comando SQL. Para restaurar os dados do sistema, quando necessário, basta rodar este script.

Toda vez que efetuar o backup, crie um novo arquivo, identificando-o com a data atual em seu nome, conforme o exemplo anterior. Para identificar os backups, utilizamos um formato de data americano, sem o uso de caracteres especiais, para permitir uma ordem natural dos arquivos, do mais antigo ao mais recente. Mantenha pelo menos uns 30 backups do SGBD, visto que os arquivos criados pelo **mysql-dump** estão em formato texto, por isso, podem ser facilmente compactados posteriormente!

Capítulo 36

A função de um DBA (*DataBase Administrator*)

O termo DBA surgiu em função das necessidades das empresas em ter disponível uma equipe disposta exclusivamente a cuidar do SGBD da empresa. Mas, por que cuidar exclusivamente do SGBD de uma empresa é tão importante?

Podemos dizer que os dados de uma empresa são um de seus principais patrimônios. Visto que, ao longo dos anos, uma empresa pode acumular em seu banco de dados milhões de clientes, incluindo nestes dados não apenas os dados pessoais destes clientes, mas também um histórico de todas as operações financeiras desta empresa. Por estes e outros motivos, podemos dizer que estes dados devem ser muito bem guardados e protegidos contra acessos indevidos e falhas lógicas ou mecânicas. Os dados, normalmente, referem-se a milhares ou milhões de pessoas, físicas ou jurídicas.

Não há nada mais desagradável do que uma empresa dizer para a gente que “Andou perdendo dados, por isso, será necessário re cadastrar!”. Este tipo de situação nos traz uma impressão muito ruim, pois sabemos que, se aquela empresa “perdeu” dados, é porque não estava bem preparada para mantê-los em seus computadores! O pior de tudo isso é não sabermos “para quem” estes dados foram perdidos. Nesta situação, a imagem da empresa já não será mais a mesma, e isto não tem volta!

A função de um DBA é definir estruturas de dados para as empresas e providenciar todo tipo de rotina e método para manter a integridade e segurança destes dados, além de impedir redundâncias ou definição de valores não necessários ou irrelevantes ao SGBD.

O papel de um DBA inicia-se em uma equipe de Análise e Desenvolvimento de Software. Um analista de sistemas deve comunicar-se com um DBA antes de definir a estrutura dos dados do sistema. Isto não quer dizer que o DBA criará a estrutura de dados para o analista de sistemas, pois tal estrutura depende da análise de requisitos. Porém, ele deve participar da definição, para impedir incompatibilidade com o SGBD adotado e, principalmente, contribuir com rotinas de gerenciamento dos dados que deverão estar na camada de Negócio do modelo.

Após a implantação do sistema, o DBA irá continuar com suas atividades de gerenciamento sobre o SGBD. Ele deve ter acesso à equipe de análise para possíveis mudanças futuras na segurança dos dados do sistema. As principais atividades de um DBA, após a implantação de um sistema, estão logo a seguir:

- criar planos e rotinas de backups do SGBD, em vários locais físicos diferentes e com total segurança na integridade e acesso dos dados;
- criar e gerenciar planos e rotinas para manter a segurança física e lógica dos dados;
- criar e gerenciar planos e rotinas para manter a segurança de acesso aos dados, impedindo o acesso indevido a pessoas não-autorizadas. Isto deve ser feito em conjunto com os analistas de sistemas;
- elaborar rotinas para checar se a segurança no acesso dos dados está sendo obedecida;
- efetuar a manutenção constante dos dados, buscando uma melhor performance do SGBD, sem deixar de lado a alta segurança no acesso e integridade dos dados;
- não permitir redundância nos dados;
- desfragmentação e reparação de tabelas dos bancos de dados;
- administrar senhas de acesso ao banco de dados, promovendo mudanças constantes de senhas de usuários e uso de senhas seguras;
- colaborar com a equipe da camada de Negócio, do modelo de desenvolvimento em N camadas;
- ter e exigir ética profissional, no acesso ao SGBD;
- elaborar e manter plano de criptografia de dados com uso de chave privada, para tráfego na Ethernet e Internet;
- manter dispositivos de segurança, como Servidor Proxy e *Firewall*, antes do acesso ao Servidor de Banco de Dados.

Manter a integridade e segurança dos dados de uma empresa é muito importante. Isso deve ser feito por uma equipe de profissionais dedicados, os DBA's! Se você quer ser um DBA, lembre-se de que o trabalho é difícil, mas é gratificante estar ligado à proteção e manutenção de um dos maiores patrimônios da empresa!

Anexo

Funções em MySQL

Principais funções para *strings* e caracteres

Função	Descrição
ASCII(string)	Esta função retorna o código ASCII do caractere informado. Exemplo: SELECT ASCII('2') retorna '50'.
BIN(n)	Esta função retorna uma representação em string de um número. Exemplo: SELECT BIN(12) retorna '1100'.
CHAR(N,... [USING charset_name])	Esta função retorna os caracteres correspondentes a cada número inteiro passado (no caso, código ASCII). Exemplo: SELECT CHAR(77,121,83,81,76) retorna 'MySQL'.
CONCAT_WS(separator,string1,string2,...)	Retorna uma série de strings concatenadas com separador. Exemplo: SELECT CONCAT_WS('-', 'A', 'B', 'C') retorna 'A-B-C'.
CONCAT(string1, string2, ...)	Retorna uma série de strings concatenadas. Exemplo: SELECT CONCAT('My', 'S', 'QL') retorna 'MySQL'.
ELT(n, string1, string2, string3, ...)	Esta função retorna a string no índice (posição), indicado no primeiro parâmetro. Exemplos: SELECT ELT(1, 'ab', 'cd', 'ef', 'gh') retorna 'ab' e SELECT ELT(4, 'ab', 'cd', 'ef', 'gh') retorna 'gh'.
FIELD(str, string1, string2, string3, ...)	Esta função retorna o índice (posição) do primeiro parâmetro dentro da sequência de itens. Exemplo: SELECT FIELD('cd', 'ab', 'cd', 'ef', 'gh') retorna '2'.
FORMAT(n,d)	Esta função retorna um número formatado com o número de casas decimais informado. Exemplos: SELECT FORMAT(122.1,4) retorna '122.1000', e SELECT FORMAT(122.1,0) retorna '122'.
HEX(numérico ou string)	Esta função retorna a representação decimal de um número ou string. Exemplos: SELECT HEX(255) retorna 'FF' e SELECT HEX('abc') retorna '616263'.

Função	Descrição
INSERT(string, posição, comprimento, substring)	Esta função insere uma substring em uma posição específica até o número de caracteres informado. Exemplo: SELECT INSERT('AAAAAAA', 3, 4, 'BBBB') retorna 'AABBBBAA'.
INSTR(string, substring)	Esta função retorna a posição da primeira ocorrência de uma substring em uma string. Exemplo: SELECT INSTR('ABCDEDE', 'DE') retorna '4'.
LEFT(string, comprimento)	Esta função retorna os n caracteres à esquerda da string. Exemplo: SELECT LEFT('abcdefgh', 5) retorna 'abcde'.
LENGTH(string)	Esta função retorna o tamanho de uma string em bytes. Exemplo: SELECT LENGTH('texto') retorna '5'.
LOCATE(substring, string[, posição])	Esta função retorna a posição da primeira ocorrência (ou a partir da posição definida) de uma substring em uma string. Exemplos: SELECT LOCATE('FG','ABCDE') retorna 0 e SELECT LOCATE('DE', 'ABCDEDE', 5) retorna '6'.
LOWER(string)	Esta função retorna uma string toda em letras minúsculas. Exemplo: SELECT LOWER('MYSQL') retorna 'mysql'.
LPAD(string, comprimento, string de preenchimento)	Esta função retorna uma string preenchida à esquerda com o caractere ou string definida no segundo parâmetro. Exemplos: SELECT LPAD('ab',4,'11') retorna '11ab' e SELECT LPAD('ab',1,'11') retorna 'a'.
LTRIM(string)	Esta função remove os espaços em branco à esquerda da string. Exemplo: SELECT LTRIM(' abcdef') retorna 'abcdef'.
MID(string, posição, comprimento)	Retorna uma substring a partir da posição e com comprimento definidos. Função da mesma forma que a função SUBSTRING.
REPEAT(string,n)	Esta função retorna a repetição de uma string n vezes. Exemplo: SELECT REPEAT('MySQL', 3) retorna 'MySQLMySQLMySQL'.

Função	Descrição
REPLACE(string, string a ser substituída, string de substituição)	Esta função substitui as ocorrências de uma string por outra, definidas como parâmetro. Exemplo: SELECT REPLACE('www.mysql.com', 'm', '-') retorna 'www.-ysql.co-'. Esta função inverte a ordem dos caracteres de uma string. Exemplo: SELECT REVERSE('abc') retorna 'cba'.
REVERSE(string)	
RIGHT(string, comprimento)	Esta função retorna os n caracteres à direita da string. Exemplo: SELECT RIGHT('abcdef', 4) retorna 'cdef'.
RPAD(string, comprimento, string de preenchimento)	Esta função retorna uma string preenchida à direita com o caractere ou string definido no segundo parâmetro. Exemplos: SELECT READ('ab',4,'11') retorna 'ab11' e SELECT READ('ab',1,'11') retorna '1'.
RTRIM(string)	Esta função remove os espaços em branco à esquerda da string. Exemplo: SELECT RTRIM('abcdef ') retorna 'abcdef'.
SPACE(n)	Esta função retorna uma string com um número definido de espaços em branco. Exemplo: SELECT SPACE(6) retorna ' '.
SUBSTR(string, posição), SUBSTR(string FROM posição), SUBSTR(string, posição, comprimento), SUBSTR(string FROM posição FOR comprimento)	Estas funções retornam uma substring a partir de uma string, considerando a posição inicial e o comprimento definido. Exemplos: - SELECT SUBSTRING('www.mysql.com',5) retorna 'mysql.com'; - SELECT SUBSTRING('www.mysql.com' FROM 4) retorna 'mysql.com';
UPPER(string)	Esta função converte uma string para caixa alta (letras maiúsculas). Exemplo: SELECT UPPER('Sql') retorna 'SQL'.

Anexo.1.

Principais funções para data |||||

Função	Descrição
<code>CURDATE()</code>	Esta função retorna a data corrente.
<code>CURTIME()</code>	Esta função retorna a hora corrente.
<code>DATE _ ADD(data,INTERVAL unidade)</code>	Esta função adiciona um determinado intervalo a uma data. Ex: <code>DATE _ ADD('2007-12-31 23:59:59',INTERVAL 1 SECOND)</code> retorna <code>'2008-01-01 00:00:00'</code> .
<code>DATE _ FORMAT(data,formato)</code>	Esta função configura uma data conforme o formato especificado. Exemplo: <code>SELECT DATE _ FORMAT('2008-01-12 19:01:00', '%W %M %Y')</code> retorna <code>'Saturday January 2008'</code> e <code>SELECT DATE _ FORMAT('2008-01-12 19:01:00', '%D %y %a %d %m %b')</code> retorna <code>'12th 08 Sat 12 01 Jan'</code> .
<code>DATE _ SUB(data,INTERVAL unidade)</code>	Esta função subtrai um determinado intervalo de uma data. Ver exemplo de <code>DATE _ ADD()</code> .
<code>DATEDIFF(expressão1,expressão2)</code>	Esta função retorna a subtração de duas datas. Exemplos: <code>SELECT DATEDIFF('2008-01-12 23:59:59','2008-01-11 23:59:59')</code> retorna <code>'1'</code> e <code>'SELECT DATEDIFF('2008-01-01 00:00:00','2008-01-12 00:00:00')</code> retorna <code>'-11'</code> .
<code>DAYNAME(data)</code>	Esta função retorna o dia da semana da data informada. Exemplo: <code>SELECT DAYNAME('2008-01-12')</code> retorna <code>'Saturday'</code> .
<code>DAYOFMONTH(data)</code> <code>DAYOFWEEK(data)</code>	Esta função retorna o dia do mês. Exemplo: <code>SELECT DAYOFMONTH('2008-01-12')</code> retorna <code>12</code> . Esta função retorna o índice do dia da semana correspondente à data informada (1 é Domingo, 2 é Segunda-feira, 7 é Sábado etc.). Exemplo: <code>SELECT DAYOFWEEK('2008-01-12')</code> retorna <code>'7'</code> .
<code>DAYOFYEAR(data)</code>	Esta função retorna o dia do ano correspondente à data informada. Exemplo: <code>SELECT DAYOFYEAR('2008-02-03')</code> retorna <code>'34'</code> .
<code>EXTRACT(unidade FROM data)</code>	Esta função extrai uma parte de uma data. Exemplo: <code>SELECT EXTRACT(YEAR FROM '2008-01-12')</code> retorna <code>2008</code> .

<code>FROM _ DAYS (n)</code>	Esta função converte o número de um dia em uma data equivalente. Exemplo: <code>SELECT FROM _ DAYS(729670)</code> retorna <code>'1997-10-08'</code>.
<code>GET _ FORMAT([DATE TIME DATETIME], ['EUR' 'USA' 'JIS' 'ISO' 'INTERNAL'])</code>	Esta função retorna uma string de formato de data de acordo com a localização informada. Exemplos: – <code>SELECT GET _ FORMAT (DATE, 'USA')</code> retorna <code>'%m.%d.%Y'</code>; – <code>SELECT GET _ FORMAT (DATETIME, 'USA')</code> retorna <code>'%Y-%m-%d %H.%i.%s'</code>; – <code>SELECT GET _ FORMAT (TIME, 'USA')</code> retorna <code>'%h:%i:%s %p'</code>.
<code>HOURL (hora)</code>	Esta função extrai a hora de uma data. Exemplo: <code>SELECT HOURL ('10:05:03')</code> retorna <code>'10'</code>.
<code>LAST _ DAY(data)</code>	Esta função retorna o último dia do mês da data informada. Exemplos: – <code>SELECT LAST _ DAY ('2008-02-05')</code> retorna <code>'2008-02-29'</code>; <code>SELECT LAST _ DAY ('2007-02-05')</code> retorna <code>'2007-02-28'</code>; – <code>SELECT LAST _ DAY ('2008-03-32')</code> retorna <code>NULL</code>.
<code>MAKEDATE (year, dayofyear)</code>	Esta função cria uma data a partir de um ano e do dia do ano. Exemplos: – <code>SELECT MAKEDATE (2008, 31)</code> retorna <code>'2008-01-31'</code>; – <code>SELECT MAKEDATE (2007, 365)</code> retorna <code>'2007-12-31'</code>; – <code>SELECT MAKEDATE (2008, 0)</code> retorna <code>'NULL'</code>.
<code>MAKETIME (hour, minute, second)</code>	Cria uma hora a partir dos valores informados. Exemplo: <code>SELECT MAKETIME (12, 25, 30)</code> retorna <code>'12:25:30'</code>.
<code>MICROSECOND (expressão)</code>	Esta função retorna os microssegundos de uma data. Exemplos: <code>SELECT MICROSECOND ('12:00:00.12345')</code> retorna <code>12345</code> e <code>SELECT MICROSECOND ('2007-12-31 23:59:59.000010')</code> retorna <code>10</code>.
<code>MINUTE (hora)</code>	Esta função retorna os minutos de uma hora. Exemplo: <code>SELECT MINUTE ('08-02-03 10:05:03')</code>; retorna <code>5</code>.
<code>MONTH (data)</code>	Esta função retorna o mês de uma data. Exemplo: <code>SELECT MONTH ('2008-02-03')</code> retorna <code>2</code>.

<code>MONTHNAME(data)</code>	Esta função retorna o nome do mês de uma data. Exemplo: <code>SELECT MONTHNAME('2008-02-15')</code> retorna 'February'.
<code>NOW()</code>	Esta função retorna a data e a hora correntes. Exemplo: <code>SELECT NOW();</code> .
<code>SECOND(hora)</code>	Esta função retorna os segundos de uma hora (0-59). Exemplo: <code>SELECT SECOND('10:05:03')</code> retorna 3.
<code>SUBTIME(expressão1, expressão2)</code>	Esta função subtrai horas de uma data ou de outra hora. Exemplo: <code>SELECT SUBTIME('2007-12-31 23:59:59.999999', '11:1:1.000002')</code> retorna '2007-12-30 22:58:58.999997'.
<code>TIME_FORMAT(hora, formato)</code>	Esta função formata uma hora. Exemplo: <code>SELECT TIME_FORMAT('10:00:00', '%H %k %h %I %l')</code> retorna '10 10 04 04 4'.
<code>TIME(expressão)</code>	Esta função extrai a hora (com minutos e segundos) de uma expressão de data. Exemplo: <code>SELECT TIME('2008-12-31 01:02:03')</code> retorna '01:02:03'.
<code>TIMESTAMP(expressão), TIMESTAMP(expressão1, expressão2)</code>	Esta função, quando utilizada com apenas um argumento, retorna a expressão em formato data/hora. Com dois argumentos, retorna, também nesse formato, a soma das duas expressões de data. Exemplos: <code>SELECT TIMESTAMP('2003-12-31')</code> retorna '2003-12-31 00:00:00' e <code>SELECT TIMESTAMP('2007-12-31 12:00:00', '12:00:00')</code> retorna '2008-01-01 00:00:00'.
<code>TIMESTAMPADD(unidade, intervalo, expressão datetime)</code>	Esta função adiciona um intervalo à expressão datetime. Exemplos: <code>SELECT TIMESTAMPADD(MINUTE, 1, '2008-01-02')</code> retorna '2008-01-02 00:01:00' e <code>SELECT TIMESTAMPADD(WEEK, 1, '2008-01-02')</code> retorna '2008-01-09'.
<code>TIMESTAMPDIFF(unidade, expressão datetime1, expressão datetime2)</code>	Esta função subtrai um determinado intervalo de uma expressão datetime em relação a outra. Exemplos: <code>SELECT TIMESTAMPDIFF(MONTH, '2008-02-01', '2008-05-01')</code> retorna 3 e <code>SELECT TIMESTAMPDIFF(YEAR, '2008-05-01', '2007-01-01')</code> retorna '-1'.
<code>TO_DAYS(data)</code>	Esta função retorna uma determinada data convertida em dias corridos. Exemplo: <code>SELECT TO_DAYS('1997-10-06')</code> retorna '729668'.

WEEKDAY(data)	Esta função retorna o índice do dia da semana relacionado à data informada (0 é Segunda-feira, 1 é Terça-feira...). Exemplo: <code>SELECT WEEKDAY('2008-01-01 22:23:00')</code> retorna '1'.
WEEKOFYEAR(data)	Esta função retorna a semana do calendário em que se localiza a data informada. Exemplo: <code>SELECT WEEKOFYEAR('2008-01-31')</code> retorna '5'.
YEAR(data)	Esta função retorna um ano a partir de uma determinada data. Exemplo: <code>SELECT YEAR('08-02-03')</code> retorna '2008'.

Anexo.2.

Principais funções matemáticas |||||

Função	Descrição
ABS(n)	Retorna o valor absoluto.
ACOS(a)	Retorna o arco-co-seno de a.
ASIN(a)	Retorna o arco-seno de a.
ATAN2(x,y), ATAN(x,y)	Retorna o arco-tangente de x e y.
ATAN(a)	Retorna o arco-tangente de a.
CEIL(a)	Retorna o limite superior de a.
CEILING(a)	Ver CEIL().
CONV(a,base1, base2)	Converte a da base1 para base2.
COS(a)	Retorna o co-seno de a.
COT(a)	Retorna a cotangente de a.
DEGREES(a)	Converte radiano em grau.
EXP(x) FLOOR(a)	Eleva a base do logaritmo à potência de x. Retorna o limite inferior de a.
MOD(x,y)	Retorna o resto da divisão de x por y.
LN(x)	Retorna o logaritmo natural ou neperiano de x.
LOG10(x)	Retorna o logaritmo de x na base dez.
LOG2(x)	Retorna o logaritmo de x na base dois.

<code>LOG(x)</code> e <code>LOG (x,b)</code>	Retorna o logaritmo natural de <code>x</code> na base <code>b</code> .
<code>PI()</code>	Retorna o valor de <code>PI</code> .
<code>POW(x,y)</code>	Retorna o valor de <code>x</code> elevado à potência de <code>y</code> .
<code>POWER(x,y)</code>	Ver <code>POW()</code> .
<code>RADIANS(a)</code>	Retorna o grau convertido em radianos.
<code>RAND()</code>	Retorna um número randômico com ponto flutuante.
<code>ROUND(n)</code>	Arredonda o número passado.
<code>SIGN(a)</code>	Retorna o sinal do argumento (-1, 0, +1).
<code>SIN(a)</code>	Retorna o seno de <code>a</code> .
<code>SQRT(a)</code>	Retorna a raiz quadrada de <code>a</code> .
<code>TAN(a)</code>	Retorna a tangente de <code>a</code> .
<code>TRUNCATE(n,d)</code>	Trunca o número <code>n</code> em <code>d</code> casas decimais.

Anexo.3.

Principais funções de conversão |||||

Função	Descrição
<code>CONVERT_TZ(data, timezone1, timezone2)</code>	Esta função converte uma data de um time zone para outro. Exemplo: <code>SELECT CONVERT_TZ('2008-01-01 12:00:00', '+00:00', '+05:00')</code> retorna '2008-01-01 17:00:00'.
<code>SEC_TO_TIME(seconds)</code>	Esta função converte segundos para o formato 'HH:MM:SS'. Exemplo: <code>SELECT SEC_TO_TIME(2378)</code> retorna '00:39:38'.
<code>STR_TO_DATE(string,formato da string)</code>	Esta função converte uma string em data. Exemplos: <code>SELECT STR_TO_DATE('00/00/0000', '%m/%d/%Y')</code> retorna '0000-00-00' e <code>SELECT STR_TO_DATE('04/31/2004', '%m/%d/%Y')</code> retorna '2004-04-31'.
<code>TIME_TO_SEC(time)</code>	Esta função converte uma hora em segundos. Exemplo: <code>SELECT TIME_TO_SEC('22:23:00')</code> retorna '80580' e <code>SELECT TIME_TO_SEC('00:39:38')</code> retorna '2378'.

Anexo.4.

